
DATA BASE MANAGEMENT SYSTEM

Verified by:

Prof. Amogh Deshmukh

Prepared by:

Shreelaxmi k

1. Lab Instructions:

- Follow all instructions carefully in the lab.
- Without permission mobile phones are not allowed.
- Formals, shoes and ID Cards are compulsory for the lab.
- Don't install or delete any software without permission.
- Students should complete writing their experiment records before arriving at the laboratory and have them reviewed and signed by the Lab Instructor.

2. Lab Exam Instructions and Evaluation:

- A total of 12 experiments, as outlined in the provided manual, collectively account for 100 marks.
- There will be two separate examinations. The first examination will cover the first 6 experiments, and the second examination will assess the remaining 6 experiments. By default, these examinations will take place in the week following the completion of each set of 6 experiments.
- During the examination, a single experiment will be randomly selected from these six for the exam evaluation.
- Each examination carries a total of 50 marks, distributed as follows:

3. Course Objectives:

- To understand the fundamentals of database design, including schema creation and attribute identification.
- To develop proficiency in SQL for performing basic CRUD (Create, Read, Update, Delete) operations.
- To explore advanced SQL queries using clauses such as DISTINCT, WHERE, ORDER BY, and functions like COUNT, MIN, MAX, SUM, and AVG.
- To learn and apply constraints like PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and DEFAULT for maintaining data integrity.
- To master JOIN operations (INNER, LEFT, RIGHT, OUTER) for combining and analyzing data across tables.
- To perform data organization and filtering using GROUP BY, HAVING, and advanced WHERE clauses with operators.
- To create and utilize database views and stored procedures for simplifying and automating tasks.
- To practice solving real-world database problems with scenario-based exercises.
- To introduce NoSQL concepts and gain hands-on experience with MongoDB for document-based data management.
- To execute advanced MongoDB queries, updates, and aggregations for complex data handling.
- To understand and apply indexing and search techniques to enhance database performance.

4. Course Learning Outcomes:

CILO 1	Understand and apply the knowledge of databases to E-R modeling and Relational Model.
CILO 2	Design a normalized database and be able to perform transaction management and concurrency control.
CILO 3	Explore various database architectures.
CILO 4	Create a sample database for any specific application.

5. Module:

Module No.	Topics	CILO Mapping
Unit 1	Introduction to Database Management Systems	CILO 1
Unit 2	Database Design	CILO 2
Unit 3	Transaction Management and Storage	CILO 2

DBMS Laboratory Manual,

School Of Technology

Unit 4	NoSQL Database	CILO 3
Unit 5	Data Replication	CILO 4

6. Textbooks:

[T1] Database Systems Concepts, 6th edition by Abraham Silberschatz, Henry F. Krth, S. Sudarshan, McGraw Hill.

[T2] Computer System Architecture, M. Morris Mano, Pearson.

7. Reference Books:

[R1] Principles of Database and Knowledge Base Systems, Vol 1 by J.D. Ullman, Computer Science Press.

[R2] Fundamentals of Database Systems, 5th Edition by R. Elmasri and S. Navathe, Pearson Education.

[R3] Foundations of Databases, Reprint by Serge Abiteboul, Richard Hull, Victor Vianu, Addison-Wesley.

8. E-content:

1. <https://www.coursera.org/learn/database-management>
2. <https://www.coursera.org/learn/introduction-to-nosql-databases>

9. Attendance Policy:

1. A Student must normally maintain a minimum of 75% attendance in the course.
2. Lab without which he/she shall be disqualified from appearing in the respective Lab examination`

Evaluation Scheme:

Lab Exam – I:

Maximum Marks: 45 Marks

Sr.no	Marks of Distribution	Marks
1	Performed Experiment	35
2	Continuous Attendance	05
3	Viva	05

Lab Exam – II:

Maximum Marks: 55 Marks

Sr.no	Marks of Distribution	Marks
1	Performed Experiment	35
2	Record Book	10
3	Continuous Attendance	05
4	Viva	05

INDEX

S.no	Experiment Names	Page no	CIO Mapping
1	Identification of attributes based on a given scenario.	5 - 7	CLO 1
2	Introduction to SQL queries, performing basic CRUD operations: (31 Jul – 5 Aug) a. Create DB b. Create Table c. Insert data into table d. Fetch data from table (Select query and basic where clause) e. Update records in table f. Delete table and DB	8 - 9	CLO 1
3	More SQL Queries: (7-12 Aug) a. Select DISTINCT b. Where with AND, OR and NOT c. Where with BETWEEN d. ORDERBY ASC DESC e. Where Clause with ISNULL f. Select TOP g. MIN(), MAX(), AVG(), SUM() and COUNT()	10 - 15	CLO 1
4	ALTER Table and applying constraints	16 - 28	CLO 2
5	ALIASES and JOINS - a. AS Keyword b. JOIN c. INNER JOIN d. LEFT JOIN e. RIGHT JOIN f. OUTER JOIN g. UNION	29 - 36	CLO 2
6	LIKE, GROUPBY, HAVING, EXISTS, ANY, ALL	37 - 46	CLO 3
7	VIEW and Procedures	47 - 48	CLO 3
8	Scenario based question on MySQL for practice	49 - 51	CLO 3
9	Introduction to MongoDB and Performing CRUD operations a. Create DB b. Create Collection c. InsertOne and InertMany d. Find and FindOne e. UpdateOne and UpdateMany f. DeleteOne and DeleteMany	52 – 60	CLO 4
10	MongoDB Query Operators (2-7 Oct)	61 - 78	CLO 4
11	MongoDB Update Operations - a. Fields b. Array	79 - 90	CLO 4
12	MongoDb Aggregations.	91 - 102	CLO 4
13	Search and Index	103 - 106	CLO 4

Experiment No: 1

1. Objective: Identification of attributes based on a given scenario.

What are all attributes required to create a Database for any hotel booking?

Creating a database for a hotel booking system requires careful consideration of various attributes to efficiently manage the data related to reservations, rooms, guests, and other relevant information. Here are some essential attributes that you might need for a hotel booking database:

1. Guest Information:

- Guest ID (unique identifier)
- First Name
- Last Name
- Contact Information (phone number, email)
- Address

2. Room Information:

- Room ID (unique identifier)
- Room Type (e.g., single, double, suite)
- Room Number
- Room Rate
- Room Capacity (number of guests it can accommodate)
- Description

3. Reservation Information:

- Reservation ID (unique identifier)
- Guest ID (foreign key to link with guest information)
- Room ID (foreign key to link with room information)
- Check-in Date
- Check-out Date
- Total Price
- Special Requests

4. Payment Information:

- Payment ID (unique identifier)
- Reservation ID (foreign key to link with reservation information)
- Payment Date
- Payment Method (e.g., credit card, cash)
- Amount Paid

5. Staff Information:

- Staff ID (unique identifier)
- Staff Name
- Job Title
- Contact Information

6. Amenities:

- Amenities ID (unique identifier)
- Amenity Name
- Description - Price (if applicable)

7. Booking History:

- Booking History ID (unique identifier)
- Guest ID (foreign key to link with guest information)
- Reservation ID (foreign key to link with reservation information)
- Dates of Previous Stays
- Total Amount Spent

8. Availability Calendar:

- Date
- Room ID (foreign key to link with room information)
- Availability Status (e.g., available, booked)

9. User Accounts (for staff access):

- User ID (unique identifier)
- Username
- Password
- Access Level (e.g., admin, receptionist, manager)

10. Reviews and Ratings:

- Review ID (unique identifier)
- Guest ID (foreign key to link with guest information)
- Room ID (foreign key to link with room information)
- Rating
- Review Text
- Date of Review

11. Promotions and Discounts:

- Promotion ID (unique identifier)
- Promotion Name
- Discount Percentage
- Valid Dates

12. Hotel Information:

- Hotel Name
- Location
- Contact Information
- Check-in/Check-out Policies

Experiment No: 2

1. Objective: Introduction to SQL queries, performing basic CRUD operations:

- a** Select DISTINCT
- b** Where with AND, OR and NOT
- c** Where with BETWEEN
- d** ORDERBY ASC|DESC
- e** Where Clause with ISNULL
- f** Select TOP
- g** MIN(), MAX(), AVG(), SUM() and COUNT()

a. Create DataBase -

⇒ create database Students;
(CREATE DATABASE database_name;)

b. Create Table -

⇒ create table Students(Name varchar(20), ID numeric(4), Course varchar(8), DOB numeric(6), Gender varchar(2), BloodGroup varchar(4));
(CREATE TABLE table_name(<columnname1> <datatype1>,
 <columnname2> <datatype2>,.....);)

c. Insert data into table -

⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Abhiram', 1001, 'CSE-AIML', 101003, 'M', 'O+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Abhimanyu', 1002, 'CSE', 010303, 'M', 'B+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Rudra', 1003, 'CSE-DS', 210503, 'M', 'O+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Nandani', 1004, 'CSE', 280303, 'F', 'B+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Ridhima', 1005, 'CSE-BIC', 031003, 'F', 'O+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Nithya', 1006, 'CSE-DS', 190603, 'F', 'A+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Rohit', 1007, 'CSE-BIC', 070203, 'M', 'A+ve');
⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Kshithija', 1008, 'CSE-BIC', 180903, 'F', 'O+ve');

- ⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Veer', 1009, 'CSE-AIML', 300503, 'M', 'B+ve');
- ⇒ insert into Students (Name, ID, Course, DOB, Gender, BloodGroup) values ('Sita', 1010, 'CSE', 291203, 'F', 'A+ve');

Number of Records: 10

Name	ID	Course	DOB	Gender	BloodGroup
Abhiram	1001	CSE-AIML	101003	M	O+ve
Abhimanyu	1002	CSE	10303	M	B+ve
Rudra	1003	CSE-DS	210503	M	O+ve
Nandani	1004	CSE	280303	F	B+ve
Ridhima	1005	CSE-BIC	31003	F	O+ve
Nithya	1006	CSE-DS	190603	F	A+ve
Rohit	1007	CSE-BIC	70203	M	A+ve
Kshithija	1008	CSE-BIC	180903	F	O+ve
Veer	1009	CSE-AIML	300503	M	B+ve
Sita	1010	CSE	291203	F	A+ve

- d. Fetch data from table (Select query and basic where clause) -** ☐ select * from Students where ID == 1005;

Number of Records: 1

Name	ID	Course	DOB	Gender	BloodGroup
Ridhima	1005	CSE-BIC	31003	F	O+ve

- e. Update records in table -**

⇒ UPDATE Students SET Course = 'CSE', BloodGroup = 'B+ve' WHERE ID = 1005;
(UPDATE *table_name* SET *column1* = *value1*, *column2* = *value2*, ... WHERE *condition*;))

Experiment No: 3

1. Objective: More SQL Queries:

- a) Select DISTINCT
- b) Where with AND, OR and NOT
- c) Where with BETWEEN
- d) ORDERBY ASC|DESC
- e) Where Clause with ISNULL
- f) Select TOP

❑ create table Student (sID numeric(8) NOT NULL, sName varchar(20), Course_Name varchar(5), Address varchar(10));

insert into Student(sID, sName, Course_Name, Address) values (1001, 'Anand', 'CSE', 'Secunderabad');

insert into Student(sID, sName, Course_Name, Address) values (1002, 'Raj', 'CSEAIML', 'Hyderabad');

insert into Student(sID, sName, Course_Name, Address) values (1003, 'Ajay', 'CSEDS', 'Sadashivapet');

insert into Student(sID, sName, Course_Name, Address) values (1004, 'Sandhya', 'CSE-BIC', 'Kurnool');

insert into Student(sID, sName, Course_Name, Address) values (1005, 'Ruhanika', 'CSE-BIC', 'Nandyal');

sID	sName	Course_Name	Address
1001	Anand	CSE	Secunderabad
1002	Raj	CSE-AIML	Hyderabad
1003	Ajay	CSE-DS	Sadashivapet
1004	Sandhya	CSE-BIC	Kurnool
1005	Ruhanika	CSE-BIC	Nandyal

❏ alter table Student add email varchar(25);

sID	sName	Course_Name	Address	email
1001	Anand	CSE	Secunderabad	null
1002	Raj	CSE-AIML	Hyderabad	null
1003	Ajay	CSE-DS	Sadashivapet	null
1004	Sandhya	CSE-BIC	Kurnool	null
1005	Ruhanika	CSE-BIC	Nandyal	null

a. Select DISTINCT –

⇒ SELECT DISTINCT Course_Name from Student;

Number of Records: 3

Course_Name
CSE-AIML
CSE-DS
CSE-BIC

b. Where with AND, OR, and NOT -

⇒ select * from Student WHERE sName == 'Sandhya' AND Address == 'Kurnool';

Number of Records: 1

sID	sName	Course_Name	Address	email
1004	Sandhya	CSE-BIC	Kurnool	null

⇒ select * from Student WHERE sName == 'Sandhya' OR Address == 'Hyderabad';

Number of Records: 2

sID	sName	Course_Name	Address	email
1002	Raj	CSE-AIML	Hyderabad	<i>null</i>
1004	Sandhya	CSE-BIC	Kurnool	<i>null</i>

⇒ select * from Student WHERE sName == 'Ajay' OR Address == 'Hyderabad'
AND NOT Course_Name = 'CSE-BIC';

Number of Records: 2

sID	sName	Course_Name	Address	email
1002	Raj	CSE-AIML	Hyderabad	<i>null</i>
1003	Ajay	CSE-DS	Sadashivapet	<i>null</i>

⇒ select * from Student WHERE sID BETWEEN 1002 AND 1005 AND NOT
Course_Name = 'CSE-DS';

Number of Records: 3

sID	sName	Course_Name	Address	email
1002	Raj	CSE-AIML	Hyderabad	<i>null</i>
1004	Sandhya	CSE-BIC	Kurnool	<i>null</i>
1005	Ruhanika	CSE-BIC	Nandyal	<i>null</i>

c. Where with BETWEEN –

⇒ select * from Student WHERE sID BETWEEN 1002 AND 1004;

Number of Records: 3

sID	sName	Course_Name	Address	email
1002	Raj	CSE-AIML	Hyderabad	<i>null</i>
1003	Ajay	CSE-DS	Sadashivapet	<i>null</i>
1004	Sandhya	CSE-BIC	Kurnool	<i>null</i>

d. ORDER BY ASC | DESC –

⇒ select * from Student ORDER BY sID ASC;

Number of Records: 5

sID	sName	Course_Name	Address	email
1001	Anand	CSE	Secunderabad	null
1002	Raj	CSE-AIML	Hyderabad	null
1003	Ajay	CSE-DS	Sadashivapet	null
1004	Sandhya	CSE-BIC	Kurnool	null
1005	Ruhanika	CSE-BIC	Nandyal	null

⇒ select * from Student ORDER BY sID DESC;

Number of Records: 5

sID	sName	Course_Name	Address	email
1005	Ruhanika	CSE-BIC	Nandyal	null
1004	Sandhya	CSE-BIC	Kurnool	null
1003	Ajay	CSE-DS	Sadashivapet	null
1002	Raj	CSE-AIML	Hyderabad	null
1001	Anand	CSE	Secunderabad	null

e. Where Clause with ISNULL –

⇒ select * from Student where sName == 'Ajay' AND Address == 'Sadashivapet' and email IS NULL;

Number of Records: 1

sID	sName	Course_Name	Address	email
1003	Ajay	CSE-DS	Sadashivapet	null

a. Select TOP –

⇒ select * from Student ORDER BY sID DESC LIMIT 3;

Number of Records: 3

sID	sName	Course_Name	Address	email
1005	Ruhanika	CSE-BIC	Nandyal	null
1004	Sandhya	CSE-BIC	Kurnool	null
1003	Ajay	CSE-DS	Sadashivapet	null

⇒ delete from Student where sName == 'Anand'; sID sName
Course_Name Address email 1002 Raj CSE-AIML
Hyderabad null

1003	Ajay	CSE-DS	Sadashivapet	null
1004	Sandhya	CSE-BIC	Kurnool	null
1005	Ruhanika	CSE-BIC	Nandyal	null

⇒ ONE MORE Method to Insert into Table:

```
INSERT INTO Student(sID, sName, Course_Name, Address) VALUES (1001,  
'Anand', 'CSE', 'Secunderabad'),  
(1002, 'Raj', 'CSE-AIML', 'Hyderabad'),  
(1003, 'Ajay', 'CSE-DS', 'Sadashivapet'),  
(1004, 'Sandhya', 'CSE-BIC', 'Kurnool'),  
(1005, 'Ruhanika', 'CSE-BIC', 'Nandyal');
```

⇒ To fetch or Display the Whole Table:

```
SELECT * FROM <table_name>;
```

⇒ To RENAME a Table:

ALTER TABLE Old_table_name RENAME TO New_table_name;

⇒ TOP:

SQL Statement:

[Get your own SQL server](#)

```
SELECT TOP 3 * FROM Customers;
```

Edit the SQL Statement, and click "Run SQL" to see the result.

[Run SQL »](#)

Result:

Number of Records: 3

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Experiment No: 4

1. **Objective:** ALTER Table and applying constraints:

- a) AUTO_INCREMENT - to perform an auto-increment
- b) NOT NULL - Ensures that a column cannot have a NULL value
- c) UNIQUE - Ensures that all values in a column are different
- d) PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- e) FOREIGN KEY - Prevents actions that would destroy links between tables
- f) CHECK - Ensures that the values in a column satisfies a specific condition
- g) DEFAULT - Sets a default value for a column if no value is specified

- **create table** for an <employee> with attributes emp_no "primary key"

emp_name job hire_date salary commission

⇒ create table Employee (emp_no numeric(6) PRIMARY KEY, emp_name varchar(20), job numeric(20), hire_date varchar(10), salary numeric(8), commission numeric(6));

- **add another col** to existing table alter table <employee> add dept_no

⇒ alter table Employee add dept_no numeric(4);

- **modify the name** of the col to some suitable name alter table <employee>

rename column hire_date to joining_date;

⇒ alter table Employee rename column hire_date to joining_date;

emp_no	emp_name	job	joining_date	salary	commission	dept_no
--------	----------	-----	--------------	--------	------------	---------

- **insert values** into the table

⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no) values (201001, 'Anand', 'Manager', '16-08-2020', 50000, 1000, 2);

- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (201002, 'Vamsi', 'Manager', '07-05-2020', 50000, 1000, 2);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (201003, 'Sanjana', 'Clerk', '21-12-2020', 20000, 2000, 4);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (201004, 'Nanditha', 'Clerk', '30-01-2020', 20000, 2000, 4);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (201005, 'Rahul', 'Salesman', '19-04-2020',
40000, 3000, 5);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (211001, 'Veer', 'Assistant', '22-03-2021', 25000, 500, 3);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (211002, 'Rudra', 'Assistant', '17-05-2021', 25000, 500, 3);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (211003, 'Kshitija', 'Analyst', '27-11-2021',
35000, 1500, 1);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (211004, 'Meera', 'Analyst', '29-09-2021', 35000, 1500, 1);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (211005, 'Rohan', 'Assistant', '11-04-2021',
25000, 500, 3);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (221001, 'Jaya', 'Manager', '08-06-2022', 50000,
1000, 2);
- ⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (221002, 'Akshay', 'Engineer', '13-12-2022',
60000, 5000, 6);

⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (221003, 'Harsha', 'Engineer', '22-02-2022',
60000, 5000, 6);

⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (221004, 'Pranav', 'Engineer', '01-10-2022',
60000, 5000, 6);

⇒ insert into Employee (emp_no, emp_name, job, joining_date, salary, commission,dept_no)
values (221005, 'Sita', 'Manager', '13-07-2022', 50000, 1000, 2);

Number of Records: 15

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201001	Anand	Manager	16-08-2020	50000	1000	2
201002	Vamsi	Manager	07-05-2020	50000	1000	2
201003	Sanjana	Clerk	21-12-2020	20000	2000	4
201004	Nanditha	Clerk	30-01-2020	20000	2000	4
201005	Rahul	Salesman	19-04-2020	40000	3000	5
211001	Veer	Assistant	22-03-2021	25000	500	3
211002	Rudra	Assistant	17-05-2021	25000	500	3
211003	Kshitija	Analyst	27-11-2021	35000	1500	1
211004	Meera	Analyst	29-09-2021	35000	1500	1
211005	Rohan	Assistant	11-04-2021	25000	500	3
221001	Jaya	Manager	08-06-2022	50000	1000	2
221002	Akshay	Engineer	13-12-2022	60000	5000	6
221003	Harsha	Engineer	22-02-2022	60000	5000	6
221004	Pranav	Engineer	01-10-2022	60000	5000	6
221005	Sita	Manager	13-07-2022	50000	1000	2

make sure you enter **some sensible values**

- enter **at least 15 rows** in order to understand what all operation can be done on the created table
- **increase salary by 500** `select salary, salary+500 from <employee>;`
⇒ `select salary, salary+500 from Employee;`

Number of Records: 15

salary	salary+500
50000	50500
50000	50500
20000	20500
20000	20500
40000	40500
25000	25500
25000	25500
35000	35500
35000	35500
25000	25500
50000	50500
60000	60500
60000	60500
60000	60500
50000	50500

- display **annual salary** of an employee with his name `select emp_name as
Employee, 12*salary as Annual_Salary from <employee>` □ `select emp_name as
Employee, 12*salary as Annual_Salary from Employee;`

Employee	Annual_Salary
Anand	600000
Vamsi	600000
Sanjana	240000
Nanditha	240000
Rahul	480000
Veer	300000
Rudra	300000
Kshitija	420000
Meera	420000
Rohan	300000
Jaya	600000
Akshay	720000
Harsha	720000
Pranav	720000
Sita	600000

- Page 22 of 110

Number of Records: 15

Employee_Details
AnandManager
VamsiManager
SanjanaClerk
NandithaClerk
RahulSalesman
VeerAssistant
RudraAssistant
KshitijaAnalyst
MeeraAnalyst
RohanAssistant
JayaManager
AkshayEngineer
HarshaEngineer
PranavEngineer
SitaManager

combine col with some meaning select emp_name||' works as '||job as Employee_Details from
<employee> □ select emp_name||' works as '||job as Employee_Details from Employee;

Number of Records: 15

Employee_Details
Anand works as Manager
Vamsi works as Manager
Sanjana works as Clerk
Nanditha works as Clerk
Rahul works as Salesman
Veer works as Assistant
Rudra works as Assistant
Kshitija works as Analyst
Meera works as Analyst
Rohan works as Assistant
Jaya works as Manager
Akshay works as Engineer
Harsha works as Engineer
Pranav works as Engineer
Sita works as Manager

- display employees **belonging to a particular department** `select * from`
 `<employee> where dept_no == 5;`
⇒ `select * from Employee where dept_no == 5;`

Number of Records: 1

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201005	Rahul	Salesman	19-04-2020	40000	3000	5

- display employees **who are managers** `select * from <employee> where job ==`
 `'manager';`
⇒ `select * from Employee where job == 'Manager';`

Number of Records: 4

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201001	Anand	Manager	16-08-2020	50000	1000	2
201002	Vamsi	Manager	07-05-2020	50000	1000	2
221001	Jaya	Manager	08-06-2022	50000	1000	2
221005	Sita	Manager	13-07-2022	50000	1000	2

display employees whose salary is between 50000 and 60000 select * from
<employee> where salary between 50000 and 60000;

⇒ select * from Employee where salary between 50000 and 60000;

Number of Records: 7

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201001	Anand	Manager	16-08-2020	50000	1000	2
201002	Vamsi	Manager	07-05-2020	50000	1000	2
221001	Jaya	Manager	08-06-2022	50000	1000	2
221002	Akshay	Engineer	13-12-2022	60000	5000	6
221003	Harsha	Engineer	22-02-2022	60000	5000	6
221004	Pranav	Engineer	01-10-2022	60000	5000	6
221005	Sita	Manager	13-07-2022	50000	1000	2

- display employees who are not managers select * from
<employee> where job != 'manager';
⇒ select * from Employee where job != 'Manager';

Number of Records: 11

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201003	Sanjana	Clerk	21-12-2020	20000	2000	4
201004	Nanditha	Clerk	30-01-2020	20000	2000	4
201005	Rahul	Salesman	19-04-2020	40000	3000	5
211001	Veer	Assistant	22-03-2021	25000	500	3
211002	Rudra	Assistant	17-05-2021	25000	500	3
211003	Kshitija	Analyst	27-11-2021	35000	1500	1
211004	Meera	Analyst	29-09-2021	35000	1500	1
211005	Rohan	Assistant	11-04-2021	25000	500	3
221002	Akshay	Engineer	13-12-2022	60000	5000	6
221003	Harsha	Engineer	22-02-2022	60000	5000	6
221004	Pranav	Engineer	01-10-2022	60000	5000	6

display employee no and name who are **doing Manager or Teacher job**

⇒ select emp_no, emp_name from Employee where job in ('Manager', 'Clerk');

Number of Records: 6

emp_no	emp_name
201001	Anand
201002	Vamsi
201003	Sanjana
201004	Nanditha
221001	Jaya
221005	Sita

- **ALTER Table and applying constraints**

⇒ alter table Employee add dept_no numeric(4);

⇒ alter table Employee rename column hire_date to joining_date;

- a. AUTO INCREMENT - to perform an auto-increment**

⇒ create table Employee(emp_no numeric(8) AUTO INCREMENT, emp_name varchar(21),job numeric(24),hire_date varchar(20),salary numeric(6),commission numeric(9));

- b. NOT NULL - Ensures that a column cannot have a NULL value**

⇒ create table Employee(emp_no numeric(8) NOT NULL, emp_name varchar(21),job numeric(24),hire_date varchar(20),salary numeric(6),commission numeric(9));

- c. UNIQUE - Ensures that all values in a column are different**

⇒ create table Employee(emp_no numeric(8) NOT NULL, emp_name varchar(21),job numeric(24),hire_date varchar(20) UNIQUE,salary numeric(6),commission numeric(9));

- d. PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table**

⇒ create table Employee(emp_no numeric(8) PRIMARY KEY, emp_name varchar(21),job numeric(24),hire_date varchar(20),salary numeric(6),commission numeric(9));

e. FOREIGN KEY - Prevents actions that would destroy links between tables.

⇒ create table Employee(emp_no numeric(8) PRIMARY KEY, emp_name varchar(21) FOREIGN KEY,job numeric(24),hire_date varchar(20),salary numeric(6),commission numeric(9));

f. CHECK - Ensures that the values in a column satisfies a specific condition.

⇒ create table Employee(emp_no numeric(8) PRIMARY KEY, emp_name varchar(21),job numeric(24),hire_date varchar(20),salary numeric(6),commission numeric(9) check salary>50000);

g. DEFAULT - Sets a default value for a column if no value is specified.

⇒ create table Employee(emp_no numeric(8) PRIMARY KEY, emp_name varchar(21),job numeric(24),hire_date varchar(20) DEFAULT,salary numeric(6),commission numeric(9));

1. display employee no and name who have m in their name

⇒ select emp_no, emp_name from Employee where emp_name like '%m%';

Number of Records: 2

emp_no	emp_name
201002	Vamsi
211004	Meera

2. display employee no whose name starts with 'a'

⇒ select emp_no from Employee where emp_name like 'a%';

Number of Records: 2

emp_no
201001
221002

3. display employee no whose name ends with 'r'

⇒ select emp_no from Employee where emp_name like '%r';

Number of Records: 1

emp_no
211001

4. display employee details in order

⇒ select * from Employee order by emp_no;

Number of Records: 15

emp_no	emp_name	job	joining_date	salary	commission	dept_no
201001	Anand	Manager	16-08-2020	50000	1000	2
201002	Vamsi	Manager	07-05-2020	50000	1000	2
201003	Sanjana	Clerk	21-12-2020	20000	2000	4
201004	Nanditha	Clerk	30-01-2020	20000	2000	4
201005	Rahul	Salesman	19-04-2020	40000	3000	5
211001	Veer	Assistant	22-03-2021	25000	500	3
211002	Rudra	Assistant	17-05-2021	25000	500	3
211003	Kshitija	Analyst	27-11-2021	35000	1500	1
211004	Meera	Analyst	29-09-2021	35000	1500	1
211005	Rohan	Assistant	11-04-2021	25000	500	3
221001	Jaya	Manager	08-06-2022	50000	1000	2
221002	Akshay	Engineer	13-12-2022	60000	5000	6
221003	Harsha	Engineer	22-02-2022	60000	5000	6
221004	Pranav	Engineer	01-10-2022	60000	5000	6
221005	Sita	Manager	13-07-2022	50000	1000	2

5. display employee details in descending order

⇒ select * from Employee order by emp_no desc;

Number of Records: 15

emp_no	emp_name	job	joining_date	salary	commission	dept_no
221005	Sita	Manager	13-07-2022	50000	1000	2
221004	Pranav	Engineer	01-10-2022	60000	5000	6
221003	Harsha	Engineer	22-02-2022	60000	5000	6
221002	Akshay	Engineer	13-12-2022	60000	5000	6
221001	Jaya	Manager	08-06-2022	50000	1000	2
211005	Rohan	Assistant	11-04-2021	25000	500	3
211004	Meera	Analyst	29-09-2021	35000	1500	1
211003	Kshitija	Analyst	27-11-2021	35000	1500	1
211002	Rudra	Assistant	17-05-2021	25000	500	3
211001	Veer	Assistant	22-03-2021	25000	500	3
201005	Rahul	Salesman	19-04-2020	40000	3000	5
201004	Nanditha	Clerk	30-01-2020	20000	2000	4
201003	Sanjana	Clerk	21-12-2020	20000	2000	4
201002	Vamsi	Manager	07-05-2020	50000	1000	2
201001	Anand	Manager	16-08-2020	50000	1000	2

6. MIN() -

⇒ SELECT MIN(salary) AS min_salary FROM Employee;

Number of Records: 1

min_salary
20000

(selects the 'minimum' salary from the table)

7. MAX() -

⇒ SELECT MAX(salary) AS max_salary FROM Employee;

Number of Records: 1

max_salary
60000

(selects the 'maximum' salary from the table)

8. AVG() -

⇒ SELECT AVG(salary) AS avg_salary FROM Employee;

Number of Records: 1

avg_salary
40333.333333333336

(selects the 'average' salary from the table)

9. LIMIT -

⇒ SELECT emp_name, salary FROM Employee ORDER BY salary DESC LIMIT 1;

Number of Records: 1

emp_name	salary
Akshay	60000

10. COUNT() -

⇒ select COUNT(emp_name) as total_employees from Employee;

Number of Records: 1

total_employees
15

(after choosing the column name, in general 'count'

displays the 'total number of records' stored in the table)

11. SUM() -

⇒ select SUM(salary) as sum_salary from Employee;

Experiment No: 5

1. **Objective:** ALIASES and JOINS :

- a) AS Keyword
- b) JOIN
- c) INNER JOIN
- d) LEFT JOIN
- e) RIGHT JOIN
- f) OUTER JOIN
- g) UNION

a. **(Alias) AS Keyword** – SQL aliases are used to give a table, or a column in a table, a temporary name. Aliases are often used to make column names more readable. An alias only exists for the duration of that query. An alias is created with the ‘AS’ keyword.

• *SELECT column_name AS alias_name FROM table_name;*

⇒ *SELECT Name AS Student_Name FROM Students;*

Number of Records: 10

Student_Name
Abhiram
Abhimanyu
Rudra
Nandani
Ridhima
Nithya
Rohit
Kshithija
Veer
Sita

• *SELECT column_name(s) FROM table_name AS alias_name;*

⇒ *SELECT ID AS Student_ID, BloodGroup AS [Blood Group] FROM Students;*

(The above statement creates two aliases, one for the ID column and one for the BloodGroup column. It requires double quotation marks or square brackets if the alias name contains spaces)

Number of Records: 10

Student_ID	Blood Group
1001	O+ve
1002	B+ve
1003	O+ve
1004	B+ve
1005	B+ve
1006	A+ve
1007	A+ve
1008	O+ve
1009	B+ve
1010	A+ve

b. JOIN – A ‘Join’ clause is used to combine rows from two or more tables, based on a related column between them.

```
⇒ SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers
ON Orders.CustomerID=Customers.CustomerID;
```

Number of Records: 196

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	9/18/1996
10365	Antonio Moreno Taquería	11/27/1996
10383	Around the Horn	12/16/1996
10355	Around the Horn	11/15/1996
10278	Berglunds snabbköp	8/12/1996
10280	Berglunds snabbköp	8/14/1996
10384	Berglunds snabbköp	12/16/1996
10436	Blondel père et fils	2/5/1997

c. INNER JOIN - Returns records that have matching values in both tables. The INNER JOIN keyword selects records that have matching values in both tables.

```
• SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

⇒ select OrderID, CustomerName from Orders inner join Customers on
Orders.CustomerID = Customers.CustomerID;

Number of Records: 196

OrderID	CustomerName
10248	Wilman Kala
10249	Tradição Hipermercados
10250	Hanari Carnes
10251	Victuailles en stock
10252	Suprêmes délices
10253	Hanari Carnes
10254	Chop-suey Chinese
10255	Richter Supermarkt
10256	Wellington Importadora
10257	HILARIÓN-Abastos

d. LEFT (OUTER) JOIN - The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

- ```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```
- select CustomerName, OrderID from Customers Left Join Orders on Customers.CustomerID = Orders.CustomerID order by CustomerName;

Number of Records: 213

| CustomerName                       | OrderID |
|------------------------------------|---------|
| Alfreds Futterkiste                | null    |
| Ana Trujillo Emparedados y helados | 10308   |
| Antonio Moreno Taquería            | 10365   |
| Around the Horn                    | 10355   |
| Around the Horn                    | 10383   |
| B's Beverages                      | 10289   |
| Berglunds snabbköp                 | 10278   |
| Berglunds snabbköp                 | 10280   |
| Berglunds snabbköp                 | 10384   |
| Blauer See Delikatessen            | null    |
| Blondel père et fils               | 10265   |
| Blondel père et fils               | 10297   |
| Blondel père et fils               | 10360   |
| Blondel père et fils               | 10436   |

e. **RIGHT (OUTER) JOIN** - The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1). The result is 0 records from the left side, if there is no match.

- ```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```
- ```
select CustomerName, OrderID from Customers Right Join Orders on
Customers.CustomerID = Orders.CustomerID order by CustomerName;
```

Number of Records: 196

| CustomerName                       | OrderID |
|------------------------------------|---------|
| Ana Trujillo Emparedados y helados | 10308   |
| Antonio Moreno Taquería            | 10365   |
| Around the Horn                    | 10355   |
| Around the Horn                    | 10383   |
| Berglunds snabbköp                 | 10280   |
| Berglunds snabbköp                 | 10384   |
| Berglunds snabbköp                 | 10278   |
| Blondel père et fils               | 10297   |

- select OrderID, FirstName, LastName from Orders Right Join Employees on Orders.EmployeeID = Employees.EmployeeID order by OrderID;

Number of Records: 197

| OrderID | FirstName | LastName  |
|---------|-----------|-----------|
| null    | Adam      | West      |
| 10248   | Steven    | Buchanan  |
| 10249   | Michael   | Suyama    |
| 10250   | Margaret  | Peacock   |
| 10251   | Janet     | Leverling |
| 10252   | Margaret  | Peacock   |
| 10253   | Janet     | Leverling |
| 10254   | Steven    | Buchanan  |
| 10255   | Anne      | Dodsworth |
| 10256   | Janet     | Leverling |
| 10257   | Margaret  | Peacock   |
| 10258   | Nancy     | Davolio   |
| 10259   | Margaret  | Peacock   |
| 10260   | Margaret  | Peacock   |

**f. FULL (OUTER) JOIN** - The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records.

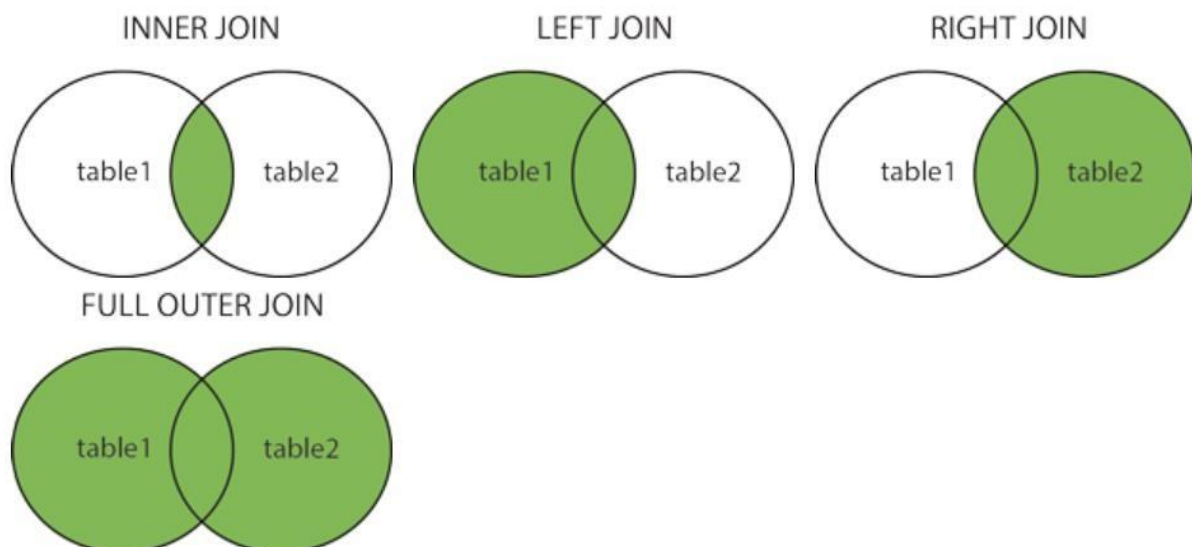
- SELECT *column\_name(s)*  
FROM *table1*  
FULL OUTER JOIN *table2*

ON *table1.column\_name* = *table2.column\_name*  
WHERE *condition*;

- select CustomerName, OrderID from Customers full join Orders on Customers.CustomerID = Orders.CustomerID order by CustomerName;

Number of Records: 213

| CustomerName                       | OrderID |
|------------------------------------|---------|
| Alfreds Futterkiste                | null    |
| Ana Trujillo Emparedados y helados | 10308   |
| Antonio Moreno Taquería            | 10365   |
| Around the Horn                    | 10355   |
| Around the Horn                    | 10383   |
| B's Beverages                      | 10289   |
| Berglunds snabbköp                 | 10278   |
| Berglunds snabbköp                 | 10280   |
| Berglunds snabbköp                 | 10384   |
| Blauer See Delikatessen            | null    |
| Blondel père et fils               | 10265   |
| Blondel père et fils               | 10297   |
| Blondel père et fils               | 10360   |
| Blondel père et fils               | 10436   |



**g. UNION** - The UNION operator is used to combine the result-set of two or more SELECT statements. The UNION operator selects only distinct values by default.

- Every SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in every SELECT statement must also be in the same order
- SELECT *column\_name(s)* FROM *table1* UNION  
SELECT *column\_name(s)* FROM *table2*;

☐ select Course from Students UNION select Course\_Name from Student;

Number of Records: 4

| Course   |
|----------|
| CSE      |
| CSE-AIML |
| CSE-BIC  |
| CSE-DS   |

**h. UNION ALL** – To allow ‘DUPLICATE’ Values, we use UNION ALL.

- SELECT *column\_name(s)* FROM *table1* UNION ALL  
SELECT *column\_name(s)* FROM *table2*;

☐ select Course from Students UNION ALL select Course\_Name from Student;

Number of Records: 14

| Course   |
|----------|
| CSE-AIML |
| CSE      |
| CSE-DS   |
| CSE      |
| CSE      |
| CSE-DS   |
| CSE-BIC  |
| CSE-BIC  |
| CSE-AIML |
| CSE      |
| CSE-AIML |
| CSE-DS   |
| CSE-BIC  |
| CSE-BIC  |

## Experiment No: 6

1. **Objective:** LIKE, GROUPBY, HAVING, EXISTS, ANY, ALL.

### Using TABLE 'Students':

Number of Records: 10

| Name      | ID   | Course   | DOB    | Gender | BloodGroup |
|-----------|------|----------|--------|--------|------------|
| Abhiram   | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Abhimanyu | 1002 | CSE      | 10303  | M      | B+ve       |
| Rudra     | 1003 | CSE-DS   | 210503 | M      | O+ve       |
| Nandani   | 1004 | CSE      | 280303 | F      | B+ve       |
| Ridhima   | 1005 | CSE      | 31003  | F      | B+ve       |
| Nithya    | 1006 | CSE-DS   | 190603 | F      | A+ve       |
| Rohit     | 1007 | CSE-BIC  | 70203  | M      | A+ve       |
| Kshithija | 1008 | CSE-BIC  | 180903 | F      | O+ve       |
| Veer      | 1009 | CSE-AIML | 300503 | M      | B+ve       |
| Sita      | 1010 | CSE      | 291203 | F      | A+ve       |

1. **LIKE** – The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the LIKE operator:

- ☐ The percent sign % represents zero, one, or multiple characters
- ☐ The underscore sign \_ represents one, single character

```
• SELECT column1, column2, ... FROM
 table_name
 WHERE columnN LIKE pattern;
```

- The \_ wildcard represents a single character. It can be any character or number, but each \_ represents one, and only one, character.
- SELECT \* FROM Students WHERE Course LIKE 'C\_\_ - I\_\_';

- Return s all the Students from a 'Course' that starts with ' C ' followed by 2 wildcard character s, then ' -' and then 1 wildcard character then followed by 'l' and then 2 wildcard characters.

Number of Records: 2

| Name    | ID   | Course   | DOB    | Gender | BloodGroup |
|---------|------|----------|--------|--------|------------|
| Abhiram | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Veer    | 1009 | CSE-AIML | 300503 | M      | B+ve       |

The % wildcard represents any number of characters, even zero characters.

- SELECT \* FROM Students WHERE Name LIKE '%t%';
- Return all Students whose 'Name' *contains* the letter 't'.

Number of Records: 4

| Name      | ID   | Course  | DOB    | Gender | BloodGroup |
|-----------|------|---------|--------|--------|------------|
| Nithya    | 1006 | CSE-DS  | 190603 | F      | A+ve       |
| Rohit     | 1007 | CSE-BIC | 70203  | M      | A+ve       |
| Kshithija | 1008 | CSE-BIC | 180903 | F      | O+ve       |
| Sita      | 1010 | CSE     | 291203 | F      | A+ve       |

- Starts With: To return records that starts with a specific letter or phrase, add the % at the end of the letter or phrase.

- SELECT \* FROM Students WHERE Name LIKE 'Ab%';
- Return all Students whose 'Name' STARTS with 'Ab'.

Number of Records: 2

| Name      | ID   | Course   | DOB    | Gender | BloodGroup |
|-----------|------|----------|--------|--------|------------|
| Abhiram   | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Abhimanyu | 1002 | CSE      | 10303  | M      | B+ve       |

- You can also combine any number of conditions using AND or OR operators.
- SELECT \* FROM Students WHERE Name LIKE 'a%' OR Name LIKE 'r%';
- Return all Students whose 'Name' starts with 'a' or starts with 'r'.

Number of Records: 5

| Name      | ID   | Course   | DOB    | Gender | BloodGroup |
|-----------|------|----------|--------|--------|------------|
| Abhiram   | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Abhimanyu | 1002 | CSE      | 10303  | M      | B+ve       |
| Rudra     | 1003 | CSE-DS   | 210503 | M      | O+ve       |
| Ridhima   | 1005 | CSE      | 31003  | F      | B+ve       |
| Rohit     | 1007 | CSE-BIC  | 70203  | M      | A+ve       |

Ends With: To return records that ends with a specific letter or phrase, add the % at the beginning of the letter or phrase.

- `SELECT * FROM Students WHERE Name LIKE '%a';` • Return all Students whose 'Name' ENDS with 'A'.

Number of Records: 5

| Name      | ID   | Course  | DOB    | Gender | BloodGroup |
|-----------|------|---------|--------|--------|------------|
| Rudra     | 1003 | CSE-DS  | 210503 | M      | O+ve       |
| Ridhima   | 1005 | CSE     | 31003  | F      | B+ve       |
| Nithya    | 1006 | CSE-DS  | 190603 | F      | A+ve       |
| Kshithija | 1008 | CSE-BIC | 180903 | F      | O+ve       |
| Sita      | 1010 | CSE     | 291203 | F      | A+ve       |

- You can also combine "starts with" and "ends with".
- `SELECT * FROM Students WHERE Name LIKE 'r%a';`
- Return all Students whose 'Name' STARTS with 'r' and ENDS with 'a'.

Number of Records: 2

| Name    | ID   | Course | DOB    | Gender | BloodGroup |
|---------|------|--------|--------|--------|------------|
| Rudra   | 1003 | CSE-DS | 210503 | M      | O+ve       |
| Ridhima | 1005 | CSE    | 31003  | F      | B+ve       |

- Contains: To return records that contains a specific letter or phrase, add the % both before and after the letter or phrase.

- `SELECT * FROM Students WHERE Course LIKE '%AIML%';`

- Return all Students whose 'Course' (name) contains the phrase 'AIML'.

Number of Records: 2

| Name    | ID   | Course   | DOB    | Gender | BloodGroup |
|---------|------|----------|--------|--------|------------|
| Abhiram | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Veer    | 1009 | CSE-AIML | 300503 | M      | B+ve       |

Combining Wildcards: Any wildcard, like % and \_ , can be used in combination with other wildcards.

- SELECT \* FROM Students WHERE Name LIKE 'r\_\_\_\_\_%';

characters in length .

Number of Records: 1

| Name    | ID   | Course | DOB   | Gender | BloodGroup |
|---------|------|--------|-------|--------|------------|
| Ridhima | 1005 | CSE    | 31003 | F      | B+ve       |

- Return all Students whose 'Name' starts with 'r' and are at least 6
  - SELECT \* FROM Students WHERE Name LIKE '\_\_\_\_i%';
- Return all Students whose 'Name' has 'i' in the 4th position.

Number of Records: 4

| Name      | ID   | Course   | DOB    | Gender | BloodGroup |
|-----------|------|----------|--------|--------|------------|
| Abhiram   | 1001 | CSE-AIML | 101003 | M      | O+ve       |
| Abhimanyu | 1002 | CSE      | 10303  | M      | B+ve       |
| Rohit     | 1007 | CSE-BIC  | 70203  | M      | A+ve       |
| Kshithija | 1008 | CSE-BIC  | 180903 | F      | O+ve       |

- Without Wildcard: If no wildcard (%) or \_ is specified, the phrase has to have an exact match to return a result.

- SELECT \* FROM Students WHERE Course LIKE 'CSE-BIC';
- Return all Students whose 'Course' is 'CSE-BIC'.

Number of Records: 2

| Name      | ID   | Course  | DOB    | Gender | BloodGroup |
|-----------|------|---------|--------|--------|------------|
| Rohit     | 1007 | CSE-BIC | 70203  | M      | A+ve       |
| Kshithija | 1008 | CSE-BIC | 180903 | F      | O+ve       |

Using TABLE ‘Employee’:

Number of Records: 15

| emp_no | emp_name | job       | joining_date | salary | commission | dept_no |
|--------|----------|-----------|--------------|--------|------------|---------|
| 201001 | Anand    | Manager   | 16-08-2020   | 50000  | 1000       | 2       |
| 201002 | Vamsi    | Manager   | 07-05-2020   | 50000  | 1000       | 2       |
| 201003 | Sanjana  | Clerk     | 21-12-2020   | 20000  | 2000       | 4       |
| 201004 | Nanditha | Clerk     | 30-01-2020   | 20000  | 2000       | 4       |
| 201005 | Rahul    | Salesman  | 19-04-2020   | 40000  | 3000       | 5       |
| 211001 | Veer     | Assistant | 22-03-2021   | 25000  | 500        | 3       |
| 211002 | Rudra    | Assistant | 17-05-2021   | 25000  | 500        | 3       |
| 211003 | Kshitija | Analyst   | 27-11-2021   | 35000  | 1500       | 1       |
| 211004 | Meera    | Analyst   | 29-09-2021   | 35000  | 1500       | 1       |
| 211005 | Rohan    | Assistant | 11-04-2021   | 25000  | 500        | 3       |
| 221001 | Jaya     | Manager   | 08-06-2022   | 50000  | 1000       | 2       |
| 221002 | Akshay   | Engineer  | 13-12-2022   | 60000  | 5000       | 6       |
| 221003 | Harsha   | Engineer  | 22-02-2022   | 60000  | 5000       | 6       |
| 221004 | Pranav   | Engineer  | 01-10-2022   | 60000  | 5000       | 6       |
| 221005 | Sita     | Manager   | 13-07-2022   | 50000  | 1000       | 2       |

2. **GROUP BY** – The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

- `SELECT column_name(s)`  
`FROM table_name`  
`WHERE condition`  
`GROUP BY column_name(s)`  
`ORDER BY column_name(s);`
- `SELECT COUNT (emp_no), job FROM Employee GROUP BY job;`
- It lists the ‘number of employees’ in ‘each job role’.

Number of Records: 6

| COUNT(emp_no) | job       |
|---------------|-----------|
| 2             | Analyst   |
| 3             | Assistant |
| 2             | Clerk     |
| 3             | Engineer  |
| 4             | Manager   |
| 1             | Salesman  |

- `SELECT COUNT(emp_no), job FROM Employee GROUP BY job ORDER BY COUNT(emp_no) DESC;`
- It lists the ‘number of employees’ in ‘each job role’ sorted high to low i.e., in Descending Order.

Number of Records: 6

| COUNT(emp_no) | job       |
|---------------|-----------|
| 4             | Manager   |
| 3             | Engineer  |
| 3             | Assistant |
| 2             | Clerk     |
| 2             | Analyst   |
| 1             | Salesman  |

3. **HAVING** - The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.

- `SELECT column_name(s)  
FROM table_name  
WHERE condition  
GROUP BY column_name(s)  
HAVING condition  
ORDER BY column_name(s);`
- `SELECT COUNT(emp_no), job FROM Employee GROUP BY job HAVING COUNT(emp_no) > 1;`
- It lists the ‘number of employees’ in each job role. Only includes ‘jobs’ with ‘more than 1 employee’.

Number of Records: 5

| COUNT(emp_no) | job       |
|---------------|-----------|
| 2             | Analyst   |
| 3             | Assistant |
| 2             | Clerk     |
| 3             | Engineer  |
| 4             | Manager   |

- `SELECT COUNT(emp_no), job FROM Employee GROUP BY job HAVING COUNT(emp_no) > 1 ORDER BY COUNT(emp_no) ASC;`
- It lists the ‘number of employees’ in ‘each job role’ sorted high to low i.e., in Descending Order AND Only includes countries with more than includes ‘jobs’ with ‘more than 1 employee’.

Number of Records: 5

| COUNT(emp_no) | job       |
|---------------|-----------|
| 2             | Analyst   |
| 2             | Clerk     |
| 3             | Assistant |
| 3             | Engineer  |
| 4             | Manager   |

4. **EXISTS** – The EXISTS operator is used to test for the existence of any record in a subquery. The EXISTS operator returns TRUE if the subquery returns one or more records.

- `SELECT column_name(s)  
FROM table_name  
WHERE EXISTS  
(SELECT column_name FROM table_name WHERE condition);`
- `SELECT SupplierName  
FROM Suppliers  
WHERE EXISTS (SELECT ProductName FROM Products WHERE  
Products.SupplierID = Suppliers.supplierID AND Price < 20);`

Number of Records: 24

| SupplierName               |
|----------------------------|
| Exotic Liquid              |
| New Orleans Cajun Delights |
| Tokyo Traders              |
| Mayumi's                   |
| Pavlova, Ltd.              |
| Specialty Biscuits, Ltd.   |
| PB Knäckebröd AB           |
| Refrescos Americanas LTDA  |

5. ANY – The ANY operator:

- returns a boolean value as a result
- returns TRUE if ANY of the subquery values meet the condition

ANY means that the condition will be true if the operation is true for any of the values in the range. The operator must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

- ```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
(SELECT column_name
FROM table_name WHERE
condition);
```
- ```
SELECT ProductName
FROM Products
WHERE ProductID = ANY (SELECT ProductID FROM OrderDetails
WHERE Quantity = 10);
```

Number of Records: 31

| ProductName                     |
|---------------------------------|
| Chais                           |
| Chang                           |
| Chef Anton's Cajun Seasoning    |
| Uncle Bob's Organic Dried Pears |
| Konbu                           |
| Tofu                            |
| Pavlova                         |
| Teatime Chocolate Biscuits      |

6. ALL - The ALL operator:

- returns a boolean value as a result
- returns TRUE if ALL of the subquery values meet the condition
- is used with SELECT, WHERE and HAVING statements

ALL means that the condition will be true only if the operation is true for all values in the range.  
The operator must be a standard comparison operator (=, <, !=, >, >=, <=, or <=).

□ ALL Syntax With SELECT –

- SELECT ALL *column\_name(s)*  
FROM *table\_name*  
WHERE *condition*;

□ ALL Syntax With WHERE or HAVING -

- SELECT *column\_name(s)*  
FROM *table\_name*  
WHERE *column\_name operator ALL*

---

```
(SELECT column_name
FROM table_name WHERE
condition);
```

- SELECT ProductName  
FROM Products  
  
WHERE ProductID = ALL (SELECT ProductID FROM OrderDetails WHERE Quantity = 10);

Number of Records: 0

| ProductName |
|-------------|
|-------------|

Experiment No: 7

1. Objective: VIEW and Procedures.

a) Views in SQL

A **View** in SQL is a virtual table created by querying data from one or more real tables. It does not store data itself but provides a way to simplify complex queries and enhance security by controlling access to specific data.

b)Procedures in SQL

A **Procedure** is a set of precompiled SQL statements that can be executed as a single unit. Procedures are used for tasks like inserting, updating, or manipulating data in the database.

You are given two tables: Employees and Departments. Your tasks involve creating specific VIEWS and PROCEDURES to fulfill certain business requirements.

Table: Employees

| EmployeeID | Name          | DepartmentID | Salary |
|------------|---------------|--------------|--------|
| 1          | John Doe      | 101          | 5000   |
| 2          | Jane Smith    | 102          | 6000   |
| 3          | Emily Davis   | 101          | 5500   |
| 4          | Michael Brown | 1031         | 7000   |
| 5          | Sarah Johnson | 102          | 7500   |
| 6          | David Miller  | 101          | 5200   |

Table: Departments

| DepartmentID | DepartmentName |
|--------------|----------------|
| 101          | Sales          |
| 102          | Marketing      |
| 103          | IT             |

Task 1: Create a VIEW to Display Employees with Salary Above a Given Threshold

Create a view named HighEarningEmployees that displays the EmployeeID, Name, Salary, and DepartmentName for employees earning more than 6000.

---

## Task 2: Create a VIEW for Total Salary Expenses per Department

Create a view named DepartmentSalaryExpenses that shows the DepartmentName and the total Salary expense for each department.

## Task 3: Create a Procedure to Add a New Employee

Create a stored procedure named AddNewEmployee that inserts a new employee into the Employees table. The procedure should take the following parameters:

- IN emp\_name VARCHAR(50)
- IN dept\_id INT
- IN emp\_salary DECIMAL(10, 2)
- CALL AddNewEmployee('Anna Taylor', 103, 6800);

## Task 4: Create a Procedure to Update Employee Salary Based on Department

Create a stored procedure named UpdateDepartmentSalary that increases the salary of all employees in a given department by a certain percentage. The procedure should take the following parameters:

- IN dept\_id INT
- IN increment\_percentage DECIMAL(5, 2)
- CALL UpdateDepartmentSalary(101, 10);

## Task 5: Create a Procedure to Remove Employees Earning Below a Certain Salary

Create a stored procedure named RemoveLowSalaryEmployees that deletes all employees who are earning below a given salary threshold. The procedure should take the following parameter:

- IN salary\_threshold DECIMAL(10, 2)
- CALL RemoveLowSalaryEmployees(6000);

## Task 6: Create a Procedure to Calculate Total Salary Expense for a Given Department

Create a stored procedure named CalculateTotalSalaryExpense that calculates the total salary expense for a given department. The procedure should take the following parameter:

- IN dept\_id INT: The DepartmentID of the department for which the total salary expense should be calculated.
- CALL TransferEmployee(3, 103);

Experiment No: 8

1. **Objective:** Scenario based question on MySQL for practice.

Employees Table

| EmployeeID | Name    | Department | Salary | Age | City     |
|------------|---------|------------|--------|-----|----------|
| 1          | Alice   | IT         | 70000  | 30  | New York |
| 2          | Bob     | IT         | 60000  | 28  | Chicago  |
| 3          | Charlie | HR         | 55000  | 35  | New York |
| 4          | David   | HR         | 48000  | 40  | Chicago  |
| 5          | Eve     | Marketing  | 70000  | 30  | Seattle  |
| 6          | Frank   | Marketing  | 75000  | 32  | Chicago  |
| 7          | Grace   | IT         | 72000  | 29  | Seattle  |

Projects Table

| ProjectID | ProjectName       | EmployeeID | Budget |
|-----------|-------------------|------------|--------|
| 1         | AI Research       | 1          | 50000  |
| 2         | Web Development   | 2          | 45000  |
| 3         | Recruitment       | 3          | 30000  |
| 4         | Employee Training | 4          | 35000  |
| 5         | Marketing Plan    | 5          | 40000  |
| 6         | Campaign          | 6          | 60000  |
| 7         | Infrastructure    | 7          | 55000  |

TASKS:

- Find the names of employees whose names end with the letter 'e'. (Use LIKE)
- Count the number of employees in each department. (Use GROUP BY)
- Find departments with an average salary greater than \$65,000. (Use HAVING)
- Find employees who are working on a project with a budget greater than \$50,000. (Use EXISTS)
- Find employees whose salary is greater than any salary in the HR department. (Use ANY)
- Find employees whose salary is greater than all salaries in the HR department. (Use ALL)
- Find employees who are from cities that start with the letter 'C'.
- Find the total budget for each employee's projects.
- Find employees who are associated with more than one project.
- List all employees who are not working on any project.
- Find projects that have a budget less than any project assigned to an employee in the 'IT' department.

12. Find employees whose salary is greater than all employees in the 'Marketing' department.

**1. Find the names of employees whose names end with the letter 'e'. (Use LIKE)**

```
SELECT Name
FROM Employees
WHERE Name LIKE '%e';
```

**2. Count the number of employees in each department. (Use GROUP BY)**

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

**3. Find departments with an average salary greater than \$65,000. (Use HAVING)**

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 65000;
```

**4. Find employees who are working on a project with a budget greater than \$50,000. (Use EXISTS)**

```
SELECT Name
FROM Employees e
WHERE EXISTS (
 SELECT 1
 FROM Projects p
 WHERE p.EmployeeID = e.EmployeeID AND p.Budget > 50000
);
```

**5. Find employees whose salary is greater than any salary in the HR department. (Use ANY)**

```
SELECT Name
FROM Employees
WHERE Salary > ANY (
 SELECT Salary
 FROM Employees
 WHERE Department = 'HR'
);
```

**6. Find employees whose salary is greater than all salaries in the HR department. (Use ALL)**

```
SELECT Name
FROM Employees
WHERE Salary > ALL (
 SELECT Salary
 FROM Employees
 WHERE Department = 'HR'
);
```

---

**7. Find employees who are from cities that start with the letter 'C'.**

```
SELECT Name
FROM Employees
WHERE City LIKE 'C%';
```

**8. Find the total budget for each employee's projects.**

```
SELECT e.Name, SUM(p.Budget) AS TotalBudget
FROM Employees e
JOIN Projects p ON e.EmployeeID = p.EmployeeID
GROUP BY e.Name;
```

**9. Find employees who are associated with more than one project.**

```
SELECT e.Name
FROM Employees e
JOIN Projects p ON e.EmployeeID = p.EmployeeID
GROUP BY e.EmployeeID, e.Name
HAVING COUNT(p.ProjectID) > 1;
```

**10. List all employees who are not working on any project.**

```
SELECT Name
FROM Employees e
WHERE NOT EXISTS (
 SELECT 1
 FROM Projects p
 WHERE p.EmployeeID = e.EmployeeID
);
```

**11. Find projects that have a budget less than any project assigned to an employee in the 'IT' department.**

```
SELECT ProjectName
FROM Projects
WHERE Budget < ANY (
 SELECT Budget
 FROM Projects p
 JOIN Employees e ON p.EmployeeID = e.EmployeeID
 WHERE e.Department = 'IT'
);
```

**12. Find employees whose salary is greater than all employees in the 'Marketing' department.**

```
SELECT Name
FROM Employees
WHERE Salary > ALL (
 SELECT Salary
 FROM Employees
 WHERE Department = 'Marketing'
);
```

## **Experiment No: 9**

1. **Objective:** Introduction to MongoDB and Performing CRUD operations.
  - a) Create DB
  - b) Create Collection
  - c) InsertOne and InertMany
  - d) Find and FindOne
  - e) UpdateOne and UpdateMany
  - f) DeleteOne and DeleteMany

### **⇒ Introduction to MongoDB**

MongoDB is a widely used NoSQL database designed to store and manage large-scale data. Unlike traditional relational databases, MongoDB uses a document-oriented data model where data is stored in flexible, schema-less JSON-like documents.

### **CRUD Operations in MongoDB**

CRUD operations represent the four core functions of managing data in MongoDB: **Create, Read, Update, and Delete.**

#### **1. Create**

The "Create" operation involves adding new documents to a collection. This allows you to initialize or expand your dataset. You can insert one or multiple documents into a collection at once.

#### **2. Read**

The "Read" operation is used to query and retrieve data from a collection. MongoDB supports a variety of query options to filter and retrieve specific subsets of documents based on criteria.

#### **3. Update**

The "Update" operation modifies existing documents in a collection. You can update one or multiple documents at a time and specify which fields to update or replace.

#### **4. Delete**

The "Delete" operation removes documents from a collection. It can be used to delete individual or multiple documents based on specified criteria.

- a. Create DB (Change or Create a Database) -**  ☐ use db\_name  
- use DBMSLab

```
> _MONGOSH
```

```
> use DBMSLab
```

```
< switched to db DBMSLab
```

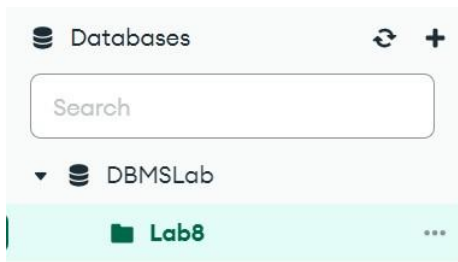
```
> use DBMSLab
```

```
< switched to db DBMSLab
```

- b. Create Collection -**

- ☐ db.createCollection("collection\_name")  
- db.createCollection("Lab8")

```
> db.createCollection("Lab8")
< { ok: 1 }
```



- c. InsertOne and InertMany (Insert Values into the Collection (Table): [Insert Documents] –**

- ☐ **InsertOne()** : to insert a single value  
- db.collection\_name.insertOne({object})  
- db.Lab8.insertOne({"id":6017, "name":"Vaishnavi"})

```
> db.Lab8.insertOne({"id":6017, "name":"Vaishnavi"})
< {
 acknowledged: true,
 insertedId: ObjectId("6525814f0736eb8de184b6f7")
}
```

```
_id: ObjectId('6525814f0736eb8de184b6f7')
id: 6017
name: "Vaishnavi"
```

## □ InsertMany() : to insert multiple values

- db.collection\_name.insertMany([ {object1}, {object2}, {...} ])
- db.Lab8.insertMany([ { "id": 6013, "name": "Vamsi" }, { "id": 5037, "name": "Mohitha" } ])

```
> db.Lab8.insertMany([{"id":6013, "name":"Vamsi"}, {"id":5037, "name":"Mohitha"}])
< {
 acknowledged: true,
 insertedIds: {
 '0': ObjectId("652581eb0736eb8de184b6f8"),
 '1': ObjectId("652581eb0736eb8de184b6f9")
 }
}
```

### DBMSLab.Lab8

0 1  
DOCUMENTS INDEXES

Documents Aggregations Schema Indexes Validation

Filter Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find </> Options

+ ADD DATA EXPORT DATA

1 - 3 of 3

```
_id: ObjectId('6525814f0736eb8de184b6f7')
id: 6017
name: "Vaishnavi"
```

```
_id: ObjectId('652581eb0736eb8de184b6f8')
id: 6013
name: "Vamsi"
```

```
_id: ObjectId('652581eb0736eb8de184b6f9')
id: 5037
name: "Mohitha"
```

```
> db.Lab8.insertMany([{"id":1001, "name":"John"}, {"id":1002, "name":"Smith"}])
< {
 acknowledged: true,
 insertedIds: {
 '0': ObjectId("65265c68e4a97f625fac3a23"),
 '1': ObjectId("65265c68e4a97f625fac3a24")
 }
}
```

```
_id: ObjectId('65265c68e4a97f625fac3a23')
id: 1001
name: "John"
```

```
_id: ObjectId('65265c68e4a97f625fac3a24')
id: 1002
name: "Smith"
```

## d. Find and FindOne –

⇒ **find()** - To select data from a collection in MongoDB. Selects all the data.

- db.collection\_name.find()
- db.Lab8.find()

```
> db.Lab8.find()
< {
 _id: ObjectId("6525814f0736eb8de184b6f7"),
 id: 6017,
 name: 'Vaishnavi'
}
{
 _id: ObjectId("652581eb0736eb8de184b6f8"),
 id: 6013,
 name: 'Vamsi'
}
{
 _id: ObjectId("652581eb0736eb8de184b6f9"),
 id: 5037,
 name: 'Mohitha'
}
```

⇒ **findOne()** - To select only one document

- db.collection\_name.findOne({sub-object})
- db.Lab8.findOne({"id":6017})

```
> db.Lab8.findOne({"id":6017})
< {
 _id: ObjectId("6525814f0736eb8de184b6f7"),
 id: 6017,
 name: 'Vaishnavi'
}
```

**e. Projection** - This parameter is an object that describes which fields to include in the results. We use a 1 to include a field and 0 to exclude a field. Notice that the `_id` field is also included. This field is always included unless specifically excluded.

- db.CollectionName.find({}, {sub-object})
- db.Lab8.find({}, {"name":1}) – “1” INCLUDES “\_id” field

```
> db.Lab8.find({}, {"name":1})
< {
 _id: ObjectId("6525814f0736eb8de184b6f7"),
 name: 'Vaishnavi'
}
{
 _id: ObjectId("652581eb0736eb8de184b6f8"),
 name: 'Vamsi'
}
{
 _id: ObjectId("652581eb0736eb8de184b6f9"),
 name: 'Mohitha'
}
```

**f. UpdateOne and UpdateMany** – The first parameter is a query object to define which document or documents should be updated. The second parameter is an object defining the updated data.

⇒ **updateOne()** - will update the first document that is found matching the provided query. \$set is used to set/update the value.

- `db.CollectionName.updateOne({reference-sub-object-search},{ $set:{subobject-to-be-updated}})`
- `db.Lab8.updateOne({"id":6017}, { $set:{ "name": "Satya Vaishnavi" }})`
- here, the reference-sub-object is "id" and update-sub-object is "name". It means that "id":6017 is to be searched and that id's "name" is to be updated.

```
> db.Lab8.updateOne({"id":6017}, { $set:{ "name": "Satya Vaishnavi" }})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: ObjectId('6525814f0736eb8de184b6f7')
id: 6017
name: "Satya Vaishnavi"
```

⇒ **updateMany() - Update all documents by 1. For this we will use the \$inc (increment) operator:**

- `db.CollectionName.updateMany({}, { $inc:{sub-object-to-be-updated:1}})`
- `db.Lab8.updateMany({}, { $inc:{ "id":1}})`

```
> db.Lab8.updateMany({}, { $inc:{ "id":1}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 5,
 modifiedCount: 5,
 upsertedCount: 0
}
```

```
_id: ObjectId('6525814f0736eb8de184b6f7')
id: 6019
name: "Satya Vaishnavi"
```

```
_id: ObjectId('652581eb0736eb8de184b6f8')
id: 6015
name: "Vamsi"
```

```
_id: ObjectId('652581eb0736eb8de184b6f9')
id: 5039
name: "Mohitha"
```

```
_id: ObjectId('65265c68e4a97f625fac3a23')
id: 1003
name: "John"
```

**g. DeleteOne and DeleteMany** - These methods accept a query object. The matching documents will be deleted.

⇒ **deleteOne()** – The deleteOne() method will delete the first document that matches the query provided.

- db.CollectionName.deleteOne({sub-object-to-be-deleted})
- db.Lab8.({"id":5037})

```
> db.Lab8.deleteOne({"id":5039})
< {
 acknowledged: true,
 deletedCount: 1
}
```

## DBMSLab.Lab8

Documents

Aggregations

Schema

Indexes

Validation

Filter



Type a query: { field: 'value' } or [Generate query](#)

+ ADD DATA

EXPORT DATA

```
_id: ObjectId('6525814f0736eb8de184b6f7')
id: 6019
name: "Satya Vaishnavi"
```

```
_id: ObjectId('652581eb0736eb8de184b6f8')
id: 6015
name: "Vamsi"
```

```
_id: ObjectId('65265c68e4a97f625fac3a23')
id: 1003
name: "John"
```

```
_id: ObjectId('65265c68e4a97f625fac3a24')
id: 1004
name: "Smith"
```

⇒ **deleteMany()** - The deleteMany() method will delete all documents that match the query provided.

- db.CollectionName.deleteMany({})
- db.Lab8.deleteMany({})

```
> db.Lab8.deleteMany({})
< {
 acknowledged: true,
 deletedCount: 4
}
```

ADD DATA

EXPORT DATA

0 - 0 of 0



This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Import Data

h. Show all Databases:

- show dbs

```
> show dbs
< admin 40.00 KiB
 config 108.00 KiB
 local 40.00 KiB
```

- Nested Lists:

□ db.CollctionName.insertOne({“subjects”: [“maths”, “physics”, “chemistry”]}))

## Experiment No: 10

### 1. Objective: MongoDB Query Operators.

```
> use DBMSLab
< switched to db DBMSLab
```

```
> db.createCollection("Lab9")
< { ok: 1 }
```

```
> db.Lab9.insertMany([{"id":1001, "name":"Arjun", "marks":70}, {"id":1002, "name":"Keerthi", "marks":80},
{"id": 1002, "name":"Rudra", "marks":75}, {"id":1004, "name":"Ridhima", "marks":60},
{"id":1005, "name":"Sanjana", "marks":50}, {"id":1006, "name":"Vaishnavi", "marks":95}, {"id":1007, "name":"Vamsi", "marks":90}])
< {
 acknowledged: true,
 insertedIds: {
 '0': ObjectId("6526c663d0b2b55b14aca5f1"),
 '1': ObjectId("6526c663d0b2b55b14aca5f2"),
 '2': ObjectId("6526c663d0b2b55b14aca5f3"),
 '3': ObjectId("6526c663d0b2b55b14aca5f4"),
 '4': ObjectId("6526c663d0b2b55b14aca5f5"),
 '5': ObjectId("6526c663d0b2b55b14aca5f6"),
 '6': ObjectId("6526c663d0b2b55b14aca5f7")
 }
}
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f1')
id: 1001
name: "Arjun"
marks: 70
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f2')
id: 1002
name: "Keerthi"
marks: 80
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f3')
id: 1003
name: "Rudra"
marks: 75
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f4')
id: 1004
name: "Ridhima"
marks: 60
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f5')
id: 1005
name: "Sanjana"
marks: 50
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f6')
id: 1006
name: "Vaishnavi"
marks: 95
```

```
_id: ObjectId('6526c663d0b2b55b14aca5f7')
id: 1007
name: "Vamsi"
marks: 90
```

- **MongoDB Query Operators** :

If the specified `<value>` is an array, MongoDB matches documents where the `<field>` matches the array exactly or the `<field>` contains an element that matches the array exactly. The order of the elements matters. `<field>` = sub-object

➤ **Comparison Operators** – These operators can be used in queries to compare values.

- **\$eq: Values are equal.** Specifies equality condition. The `$eq` operator matches documents where the value of a field equals the specified value.

- `{<field>: {$eq: <value>}}`
- `db.Lab9.find({"id":{$eq: 1006}})` is equivalent to
- `db.Lab9.find({"id": 1006})`

```
> db.Lab9.find({"id":{$eq: 1006}}) > db.Lab9.find({"id": 1006})
< { < {
 _id: ObjectId("6526c663d0b2b55b14aca5f6"), _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006, id: 1006,
 name: 'Vaishnavi', name: 'Vaishnavi',
 marks: 95 marks: 95
}
```

- **\$ne: Values are not equal.** `$ne` selects the documents where the value of the specified field is not equal to the specified value. This includes documents that do not contain the specified field.

- `{field: {$ne: <value>}}`
- `db.Lab9.find({"id": {$ne: 1006}})`

```
> db.Lab9.find({"id": {$ne: 1006}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
```

```
{
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f7"),
 id: 1007,
 name: 'Vamsi',
 marks: 90
}
```

☐ **\$gt:** Value is greater than another value.

☒ **>** the the value of the specified field is greater than (i.e.

**\$gt**

selects those documents where specified value.

- {field: {\$gt:<value>}}
- db.Lab9.find({"marks":{\$gt: 80}})

```
> db.Lab9.find({"marks":{$gt: 80}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f7"),
 id: 1007,
 name: 'Vamsi',
 marks: 90
}
```

- **\$gte: Value is greater than or equal to another value.** \$gte selects the documents where the value of the specified field is greater than or equal to (i.e. `>=`) a specified value (e.g. `value`.)

- {field: {\$gte:<value>}}
- db.Lab9.find({"marks": {\$gte: 80}})

```
> db.Lab9.find({"marks":{$gte: 80}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f7"),
 id: 1007,
 name: 'Vamsi',
 marks: 90
}
```

- **\$lt: Value is less than another value.** selects \$lt the documents where the value of the is less than (i.e. `field <`) the specified . `value`

- {field: {\$lt:<value>}} -
- db.Lab9.find({"marks":{\$lt: 80}})

```
> db.Lab9.find({"marks":{$lt: 80}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
```

- **\$lte:** Value is less than or equal to another value. \$lte selects the documents where the value of the field is less than or equal to (i.e.  $\leq$ ) the specified .

Value

- {field: {\$lte:<value>}} - db.Lab9.find({"marks":{\$lte: 80}})

```
> db.Lab9.find({"marks":{$lte: 80}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
```

- **\$in**: Value is matched within an array. The **\$in** operator selects the documents where the value of a field equals any value in the specified array. We use a 1 to include a field and 0 to exclude a field.

- {field: {\$in: [<value1>, <value2>, ... <valueN>]}}
- db.Lab9.find({"id":{\$in:[ 1001, 1005 ]}}, { \_id: 0}) – “0” EXCLUDES “\_id” field

```
> db.Lab9.find({"id":{$in:[1001, 1005]}}, { _id: 0})

< {
 id: 1001,
 name: 'Arjun',
 marks: 70
}
{
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
```

- db.Lab9.find({"id":{\$in:[ 1001, 1005 ]}}, { \_id: 1}) – “1” INCLUDES “\_id” field

```
> db.Lab9.find({"id":{$in:[1001, 1005]}}, { _id: 1})

< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1")
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5")
}
```

- db.Lab9.find({"id":{\$in:[1001, 1005]}, {}, { \_id:1}) – Displays both the data and “\_id” field

```
> db.Lab9.find({"id":{"$in:[1001, 1005]}}, {}, { _id: 1})

< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
```

➤ **Logical Operators** - These operators can logically compare multiple queries.

- ❑ **\$and**: Returns documents where both queries match. `$and` performs a logical **AND** operation on an array of *one or more* expressions (`<expression1>`, `<expression2>`, and so on) and selects the documents that satisfy *all* the expressions.

- `{ $and: [ {<expression1>}, {<expression2>}, ..., {<expressionN>} ] }`
- `db.Lab9.find({ $and: [ { "id": { $ne: 1001 } }, { "marks": { $exists: true } } ] })`

```
> db.Lab9.find({ $and: [{ "id": { $ne: 1001 } }, { "marks": { $exists: true } }] })

< {
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
```

The query selects all documents in the `Lab9` collection where:

- the `"id"` field value is not equal to `1001` and
- the `"marks"` field exists.

One more model type of query can be written with an implicit operation **AND** that combines the operator expressions for the field: -

```
“id” db.Lab9.find({"id": {$eq: 1006, $exists: true}})
```

```
> db.Lab9.find({"id": {$eq: 1006, $exists: true}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
```

---

**db.Lab9.find({\$and: [{"id": {\$ne: 1001}}, {"subject": {\$exists: false}}]})**

```
> db.Lab9.find({$and: [{"id": {$ne: 1001}}, {"subject": {$exists: false}}]})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
```

```
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
```

```
{
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f7"),
 id: 1007,
 name: 'Vamsi',
 marks: 90
}
```

The query selects all documents in the `Lab9` collection where:

- the `"id"` field value is not equal to `1001` and
- the `"subject"` field that DOESN'T exist.

□ **\$or:** Returns documents where either query matches. The operator `$or` performs a logical `OR` operation on an array of *one or more* <expressions> and selects the documents that satisfy *at least* one of the <expressions>.

- `{ $or: [ {<expression1>}, {<expression2>}, ..., {<expressionN>} ] }` - `db.Lab9.find( { $or: [ { "marks": { $lt: 70 } }, { "id": { $eq: 1006 } } ] } )`

```
> db.Lab9.find({"id": {$eq: 1006, $exists: true}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
```

- db.Lab9.find({\$or: [{"marks": {\$lt: 70}}, {"id": 1006}]})

```
> db.Lab9.find({$or: [{"marks": {$lt: 70}}, {"id": {$eq :1006}]})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
```

- This query will select all documents in the Lab9 collection where either the "marks" field value is less than 70 or the "id" field value equals 1006.

- **\$nor:** Returns documents where both queries fail to match. \$nor performs a logical NOR operation on an array of one or more query expression and selects the documents that fail all the query expressions in the array. It comes "before" the field. The \$nor has the following syntax:
  - {\$nor: [{<expression1>}, {<expression2>}, ... {<expressionN>}]}
  - db.Lab9.find({\$nor: [{"id": 1001}, {"marks": true}]})

```
> db.Lab9.find({$nor: [{"id": 1001}, {"marks": true}]})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f4"),
 id: 1004,
 name: 'Ridhima',
 marks: 60
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f5"),
 id: 1005,
 name: 'Sanjana',
 marks: 50
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f6"),
 id: 1006,
 name: 'Vaishnavi',
 marks: 95
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f7"),
 id: 1007,
 name: 'Vamsi',
 marks: 90
}
```

This query will return all documents that:

- contain the `"id"` field whose value is *not* equal to `1001` and contain the `or` `"marks"` field whose value *is not* equal to `true`
- contain the `"id"` field whose value is *not* equal to `1.99` *but do not* contain the `"marks"` field **or**
- *do not* contain the `"id"` field *but* contain the `"marks"` field whose value *is not* equal to `true` **or**
- *do not* contain the field `"id"` *and* *do not* contain the `"marks"` field

---

- `db.inventory.find({$nor: [{price: 1.99}, {qty: {$lt: 20}}, {sale: true}]})`

This query will select all documents in the collection `inventory` where:

- the `price` field value does *not* equal `1.99` **and**
- the `qty` field value is *not* less than `20` **and**
- the `sale` field value is *not* equal to `true`

including those documents that do not contain these field(s).

- 
- ☐ **\$not**: Returns documents where the query does not match. `$not` performs a logical `NOT` operation on the specified `<operator-expression>` and selects the documents that do *not* match the `<operator-expression>`. It comes "after" the field. This includes documents that do not contain the field.

- `{field: {$not: {<operator-expression>}}}`
- `db.Lab9.find({"id": {$not: {$gt: 1003}}})`

```
> db.Lab9.find({"id": {$not: {$gt: 1003 }}})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f2"),
 id: 1002,
 name: 'Keerthi',
 marks: 80
}
{
 _id: ObjectId("6526c663d0b2b55b14aca5f3"),
 id: 1003,
 name: 'Rudra',
 marks: 75
}
```

This query will select all documents in the collection `inventory` where:

- the `"id"` field value is less than or equal to `1003` or
  - the `"id"` field does not exist
- field `{ $not: { $gt: 1003 } }` is different from the `$lte` operator. `{ $lte:` exists and its

`1003}` returns *only* the documents where `"id"` value is less than or equal to . `1003`

---

- AND Variation - \$and:**

- `db.Lab9.find({$and: [{"id": {$eq: 1001}}, {"marks": {$eq: 70}}]})`

```
> db.Lab9.find({$and: [{"id": {$eq: 1001}}, {"marks": {$eq: 70}}]})
< {
 _id: ObjectId("6526c663d0b2b55b14aca5f1"),
 id: 1001,
 name: 'Arjun',
 marks: 70
}
```

---

➤ **Evaluation Operators** - The following operators assist in evaluating documents.

- ❑ **\$regex:** Allows the use of regular expressions when evaluating field values. Provides regular expression capabilities for pattern matching *strings* in queries.
- { <field>: { \$regex: /pattern/, \$options: '<options>' } }
- { <field>: { \$regex: 'pattern', \$options: '<options>' } }
- { <field>: { \$regex: /pattern/<options> } }
- In MongoDB, you can also use regular expression objects (i.e. /pattern/) to specify regular expressions: { <field>: /pattern/<options> }
- db.sampledata.find({"name":{\$regex:/^E/}})
- It will return the document where name starts with letter 'E'

```
> db.sampledata.find({"name": {$regex: /^E/}})
< {
 _id: '5f1234567890123456789015',
 employeeID: 1004,
 name: 'Emily Davis',
 age: 29,
 isEmployee: true,
 position: 'UI/UX Designer',
 salary: 70000.5,
 joinDate: '2020-06-05T00:00:00Z',
 skills: [
 'Adobe XD',
 'Sketch',
 'User Research'
],
 address: {
 street: '101 Pine St',
 city: 'Othercity',
 state: 'TX',
 zip: '98765'
 }
}
```

```
{
 _id: '5f1234567890123456789022',
 employeeID: 1011,
 name: 'Emma Hernandez',
 age: 29,
 isEmployee: true,
 position: 'Product Manager',
 salary: 92000.5,
 joinDate: '2017-02-14T00:00:00Z',
 skills: [
 'Product Roadmapping',
 'Market Research',
 'User Experience'
],
 address: {
 street: '888 Elmwood St',
 city: 'Elsewhere',
 state: 'UT',
 zip: '22345'
 }
}

{
 _id: '5f1234567890123456789031',
 employeeID: 1020,
 name: 'Ella Turner',
 age: 30,
 isEmployee: true,
 position: 'Sales Manager',
 salary: 88000.25,
 joinDate: '2008-01-20T00:00:00Z',
 skills: [
 'Sales Strategy',
 'Client Relationship',
 'Negotiation'
],
 address: {
 street: '999 Elm St',
 city: 'Nowhere',
 state: 'MA',
 zip: '99887'
 }
}
```

□ **\$text:** Performs a text search.

- `db.sampledata.createIndex({position:"text"})`
- `db.sampledata.find({$text:{$search: "Developer"}})`
- For this first you need to 'create an index: name text' and then search 'Developer' in the collection and return that document.

```
> db.sampledata.createIndex({position: "text"})
< position_text
> db.sampledata.find({$text:{$search: "Developer"}})
< {
 _id: '5f1234567890123456789013',
 employeeID: 1002,
 name: 'Jane Smith',
 age: 28,
 isEmployee: true,
 position: 'Web Developer',
 salary: 65000.75,
 joinDate: '2021-03-20T00:00:00Z',
 skills: [
 'HTML',
 'CSS',
 'JavaScript'
],
 address: {
 street: '456 Elm St',
 city: 'Sometown',
 state: 'NY',
 zip: '67890'
 }
}
{
 _id: '5f1234567890123456789019',
 employeeID: 1008,
 name: 'Ava Hall',
 age: 27,
 isEmployee: true,
 position: 'Mobile App Developer',
 salary: 72000.75,
 joinDate: '2021-01-02T00:00:00Z',
 skills: [
 'React Native',
 'Swift',
 'Android'
],
 address: {
 street: '555 Pineapple St',
 city: 'Somewhere',
 state: 'AZ',
 zip: '11234'
 }
}
```

## Experiment No: 11

### 1. Objective: MongoDB Update Operations

- a) Fields
- b) Array

#### ➤ Fields Operators - The following operators can be used to update “fields”.

We use the ‘updateOne’ or ‘updateMany’ Commands to update the fields and values.

- ❑ **\$currentDate: Sets the field value to the current date.** The \$currentDate operator sets the value of a field to the current date, either as a Date or a timestamp. The default type is Date.

- <typeSpecification> can be either:
  - a boolean true to set the field value to the current date as a Date,
  - or a document { \$type: "timestamp" } or { \$type: "date" } which explicitly specifies the type. The operator is case-sensitive and accepts only the lowercase "timestamp" or the lowercase "date". - To specify a <field> in an embedded document or in an array, use dot notation.
- { \$currentDate: { <field1>: <typeSpecification1>, ... } }
- db.sampledata.updateOne( { "name": "Emily Davis" }, { \$currentDate: { "joinDate": true } } ) - It has updated the value of 'joinDate' as the current date.

```
> db.sampledata.updateOne({"name": "Emily Davis"}, {$currentDate: {"joinDate":true}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789015"
employeeID: 1004
name: "Emily Davis"
age: 29
isEmployee: true
position: "UI/UX Designer"
salary: 70000.5
joinDate: 2023-11-03T11:04:38.482+00:00
▶ skills: Array (3)
▶ address: Object
```

- **\$inc: Increments the field value.** The \$inc operator increments a field by a specified value.
  - { \$inc: { <field1>: <amount1>, <field2>: <amount2>, ... } }
  - The following updateOne() operation uses the \$inc operator to:
    - increase the 'age' field by -1 (which decreases age)
    - increase the 'salary' field by 5000
  - db.sampledata.updateOne({"name": "Emily Davis"}, { \$inc: { "age": -1, "salary": 5000 } })

```
> db.sampledata.updateOne({"name": "Emily Davis"}, {$currentDate: {"joinDate":true}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
> db.sampledata.updateOne({"name":"Emily Davis"}, {$inc:{"age": -1, "salary": 5000}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789015"
employeeID: 1004
name: "Emily Davis"
age: 28
isEmployee: true
position: "UI/UX Designer"
salary: 75000.5
joinDate: 2023-11-03T11:04:38.482+00:00
▶ skills: Array (3)
▶ address: Object
```

❑ **\$rename: Renames the field.** The \$rename operator updates the name of a field and has the following form.

- {\$rename: {<field1>: <newName1>, <field2>: <newName2>, ...}}
- db.sampledata.updateOne({name:"Emily Davis"},{\$rename:{"joinDate": "JoiningDate"}})
- It will rename the field of 'joinDate' as 'JoiningDate' for Emily Davis.

```
> db.sampledata.updateOne({"name":"Emily Davis"}, {$rename:{"joinDate":"JoiningDate"}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789015"
employeeID: 1004
name: "Emily Davis"
age: 28
isEmployee: true
position: "UI/UX Designer"
salary: 75000.5
▶ skills: Array (3)
▶ address: Object
JoiningDate: 2023-11-03T11:04:38.482+00:00
```

- ❑ **\$set: Sets the value of a field.** The \$set operator replaces the value of a field with the specified value.
- {\$set: {<field1>: <value1>, ...}}
  - db.sampledata.updateOne({"name":"Emily Davis"}, {\$set: {"name":"Emilia Davis", "salary":10000}})
  - For the 'name' which is Emily Davis it is going to be set 'salary' as '10000' and 'name' as 'Emilia Davis'.

```
> db.sampledata.updateOne({"name":"Emily Davis"}, {$set: {"name":"Emilia Davis", "salary":10000}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789015"
employeeID: 1004
name: "Emilia Davis"
age: 28
isEmployee: true
position: "UI/UX Designer"
salary: 10000
▸ skills: Array (3)
▸ address: Object
JoiningDate: 2023-11-03T11:04:38.482+00:00
```

---

- db.sampledata.updateOne({"name": "Sophia Brown"}, {\$set : {"skills": ["Python", "R", "Machine Learning", "C++"]}})
- For the 'name' which is Sophia Brown it is going to be set 'skills' as Python, R, Machine Learning, C++ and we should update by writing the new values in the [] brackets as multiple values are stored in array form [].
- This operator only sets values, but does not add values in it.
- To add extra values, we need to write the old values along with the new values we want to add like shown in the below command and example.

```
> db.sampledata.updateOne({"name":"Sophia Brown"}, {$set: {"skills":["Python","R","Machine Learning","C++"]}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
 _id: "5f1234567890123456789017"
 employeeID: 1006
 name: "Sophia Brown"
 age: 31
 isEmployee: true
 position: "Data Scientist"
 salary: 90000
 joinDate: "2017-07-20T00:00:00Z"
 ▼ skills: Array (4)
 0: "Python"
 1: "R"
 2: "Machine Learning"
 3: "C++"
 ▼ address: Object
 street: "333 Cedar St"
 city: "Othertown"
 state: "IL"
 zip: "33445"
```

- OR ELSE, If we enter the only new value we want to add, it will be 'set' to the 'new value only' like shown in the value.

```
> db.sampledata.updateOne({"name":"Sophia Brown"}, {$set: {"skills":["C++"]}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789017"
employeeID: 1006
name: "Sophia Brown"
age: 31
isEmployee: true
position: "Data Scientist"
salary: 90000
joinDate: "2017-07-20T00:00:00Z"
▼ skills: Array (1)
 0: "C++"
▼ address: Object
 street: "333 Cedar St"
 city: "Othertown"
 state: "IL"
 zip: "33445"
```

❑ **\$unset: Removes the field from the document.** The \$unset operator deletes a particular field.

- { \$unset: { <field1>: "", ... } }
- The specified value in the \$unset expression (i.e., "") does not impact the operation. It just means to remove and make it erase.
- db.sampledata.updateOne({ "name": "Emilia Davis" }, { \$unset: { "JoiningDate": "", "address": "" } })
- It will remove 'JoiningDate' (previously 'joinDate') and 'address' from the selected document.

```
> db.sampledata.updateOne({"name":"Emilia Davis"}, {$unset: {"JoiningDate":" ", "address":" "}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789015"
employeeID: 1004
name: "Emilia Davis"
age: 28
isEmployee: true
position: "UI/UX Designer"
salary: 10000
▼ skills: Array (3)
 0: "Adobe XD"
 1: "Sketch"
 2: "User Research"
```

## ➤ Array Operators - The following operators assist with updating “arrays”.

- ⇒ **\$addToSet**: Adds distinct elements to an array. The **\$addToSet** operator adds a value to an array unless the value is already present, in which case it does nothing to that array.
- ⇒ only **\$addToSet** ensures that there are no duplicate items *added* to the set and does not affect existing duplicate elements. It does not guarantee a particular ordering of elements in the modified set.
- { \$addToSet: { <field1>: <value1>, ... } }
- db.sampledata.updateOne( { "name": "Ava Hall" }, { \$addToSet: { "skills": "Java" } } ) - It will add a new value to the array skills if it is not present already.

```
> db.sampledata.updateOne({"name": "Ava Hall"}, {$addToSet: {"skills": "Java"}})
```

```
< {
```

```
 acknowledged: true,
```

```
 insertedId: null,
```

```
 matchedCount: 1,
```

```
 modifiedCount: 1,
```

```
 upsertedCount: 0
```

```
}
```

```
 _id: "5f1234567890123456789019"
```

```
 employeeID: 1008
```

```
 name: "Ava Hall"
```

```
 age: 27
```

```
 isEmployee: true
```

```
 position: "Mobile App Developer"
```

```
 salary: 72000.75
```

```
 joinDate: "2021-01-02T00:00:00Z"
```

```
 ▼ skills: Array (4)
```

```
 0: "React Native"
```

```
 1: "Swift"
```

```
 2: "Android"
```

```
 3: "Java"
```

```
 ▼ address: Object
```

```
 street: "555 Pineapple St"
```

```
 city: "Somewhere"
```

```
 state: "AZ"
```

```
 zip: "11234"
```

- **\$pop**: Removes the first or last element of an array. The operator **\$pop** removes the first or last element of an **\$pop** a value of ( **-** array. Pass to **1**)
- remove the first element of an array and ( to **1**) remove the last element in an array.
- {**\$pop**: {<field>: <-1 | 1>, ...}}
  - db.sampledata.updateOne({"name": "Ava Hall"}, {**\$pop**: {"skills": -1}})

```
> db.sampledata.updateOne({"name": "Ava Hall"}, {$pop: {"skills": -1}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
_id: "5f1234567890123456789019"
employeeID: 1008
name: "Ava Hall"
age: 27
isEmployee: true
position: "Mobile App Developer"
salary: 72000.75
joinDate: "2021-01-02T00:00:00Z"
▼ skills: Array (3)
 0: "Swift"
 1: "Android"
 2: "Java"
▼ address: Object
 street: "555 Pineapple St"
 city: "Somewhere"
 state: "AZ"
 zip: "11234"
```

- ⇒ **\$pull**: Removes all elements from an array that match the query.  
The **\$pull** operator removes from an existing array all instances of a value or values that match a specified condition.
- ⇒ If you specify a **<condition>** and the array elements are embedded documents, **\$pull** operator applies the **<condition>** as if each array element were a document in a collection. See Remove All Items That Match a Specified **\$pull** Condition With **bulkWrite()** for an example.
- ⇒ If the specified **<value>** to remove is an array, **\$pull** removes only the elements in the array that match the specified **<value>** exactly, including order.
- ⇒ If the specified **<value>** to remove is a document, **\$pull** removes only the elements in the array that have the exact same fields and values. The ordering of the fields can differ.

- `{ $pull: { <field1>: <value|condition>, <field2>: <value|condition>, ... } }` -  
`db.sampledata.updateOne({ "name": "Ava Hall"}, { $pull: { "skills": { $in: ["Swift", "Android", "Java"] } } })`

```
> db.sampledata.updateOne({ "name": "Ava Hall"}, { $pull: { "skills": { $in: ["Swift", "Android", "Java"] } } })
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
 _id: "5f1234567890123456789019"
 employeeID: 1008
 name: "Ava Hall"
 age: 27
 isEmployee: true
 position: "Mobile App Developer"
 salary: 72000.75
 joinDate: "2021-01-02T00:00:00Z"
 ▼ skills: Array (empty)
 ▼ address: Object
 street: "555 Pineapple St"
 city: "Somewhere"
 state: "AZ"
 zip: "11234"
```

- **\$push**: Adds an element to an array. The specified value to an array.

`$push` operator appends a

- `{ $push: { <field1>: <value1>, ... } }`
- `db.sampledata.updateOne({ "name": "Ava Hall"}, { $push: { "skills": ["React Native", "Swift", "Android", "JavaScript", "C++"] } })`

```
> db.sampledata.updateOne({"name": "Ava Hall"}, {$push: {"skills": ["React Native", "Swift", "Android", "JavaScript", "C++"]}})
< {
 acknowledged: true,
 insertedId: null,
 matchedCount: 1,
 modifiedCount: 1,
 upsertedCount: 0
}
```

```
 _id: "5f1234567890123456789019"
 employeeID: 1008
 name: "Ava Hall"
 age: 27
 isEmployee: true
 position: "Mobile App Developer"
 salary: 72000.75
 joinDate: "2021-01-02T00:00:00Z"
 ▼ skills: Array (1)
 ▼ 0: Array (5)
 0: "React Native"
 1: "Swift"
 2: "Android"
 3: "JavaScript"
 4: "C++"
 ▼ address: Object
 street: "555 Pineapple St"
 city: "Somewhere"
 state: "AZ"
 zip: "11234"
```

## Experiment No: 12

1. **Objective:** MongoDB Aggregations:

- a) GROUP
- b) LIMIT
- c) PROJECT
- d) SORT
- e) MATCH
- f) COUNT
- g) LOOKUP
- h) OUT
- i) ADDFIELDS

- 1. **\$group:** Groups documents by a field and performs aggregations, like SUM, on specified fields.
- 2. **\$limit:** Limits the number of documents returned in the output.
- 3. **\$project:** Selects specific fields and can create new computed fields.
- 4. **\$sort:** Sorts documents based on specified field(s) in ascending or descending order.
- 5. **\$match:** Filters documents based on a specified condition.
- 6. **\$count:** Counts the total number of documents and returns the count as a single document.
- 7. **\$lookup:** Performs a left join with another collection based on matching fields.
- 8. **\$out:** Writes the results of the aggregation pipeline to a new collection.
- 9. **\$addFields:** Adds or modifies fields in documents by computing values.

**use SalesDB**

switched to db SalesDB

```
db["sales"].find()
```

```
db.sales.insertMany([
```

```
 { item: "apple", price: 1.2, quantity: 50, status: "A" },
 { item: "banana", price: 0.8, quantity: 100, status: "A" },
 { item: "orange", price: 1.5, quantity: 75, status: "B" },
 { item: "apple", price: 1.2, quantity: 30, status: "B" },
 { item: "banana", price: 0.8, quantity: 50, status: "A" }
```

```
])
```

```
{
```

```
 acknowledged: true,
```

```
 insertedIds: {
```

```
 '0': ObjectId('672c428ea88246916737a113'),
```

```
 '1': ObjectId('672c428ea88246916737a114'),
```

```
 '2': ObjectId('672c428ea88246916737a115'),
```

```
 '3': ObjectId('672c428ea88246916737a116'),
```

```
 '4': ObjectId('672c428ea88246916737a117')
```

```
 }
```

```
}
```

```
1. db.sales.aggregate([{$group: { _id: "$item", totalQuantity: { $sum:
 "$quantity" }}}])
```

```
{
```

```
 _id: 'apple',
```

```
 totalQuantity: 80
```

```
}
```

```
{
```

```
 _id: 'orange',
```

```
 totalQuantity: 75
```

```
}

{
 _id: 'banana',
 totalQuantity: 150
}

2. db.sales.aggregate([{$limit: 5 }])

{
 _id: ObjectId('672c428ea88246916737a113'),
 item: 'apple',
 price: 1.2,
 quantity: 50,
 status: 'A'
}

{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
 price: 0.8,
 quantity: 100,
 status: 'A'
}

{
 _id: ObjectId('672c428ea88246916737a115'),
 item: 'orange',
 price: 1.5,
 quantity: 75,
 status: 'B'
}

{
```

```
_id: ObjectId('672c428ea88246916737a116'),
item: 'apple',
price: 1.2,
quantity: 30,
status: 'B'
}
```

```
{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 price: 0.8,
 quantity: 50,
 status: 'A'
}
```

3. db.sales.aggregate([{\$project: {item: 1,totalAmount: { \$multiply: ["\$price",  
"\$quantity"] }}}])

```
{
 _id: ObjectId('672c428ea88246916737a113'),
 item: 'apple',
 totalAmount: 60
}
```

```
{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
 totalAmount: 80
}
```

```
{
 _id: ObjectId('672c428ea88246916737a115'),
 item: 'orange',
 totalAmount: 112.5
}
```

```
}
{
 _id: ObjectId('672c428ea88246916737a116'),
 item: 'apple',
 totalAmount: 36
}
{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 totalAmount: 40
}
```

```
4. db.sales.aggregate([
 {
 $project: {
 item: 1,
 totalAmount: { $multiply: ["$price", "$quantity"] }
 }
 },
 { $sort: { totalAmount: -1 } }
])
{
 _id: ObjectId('672c428ea88246916737a115'),
 item: 'orange',
 totalAmount: 112.5
}
{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
```

```
totalAmount: 80
}
{
 _id: ObjectId('672c428ea88246916737a113'),
 item: 'apple',
 totalAmount: 60
}
{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 totalAmount: 40
}
{
 _id: ObjectId('672c428ea88246916737a116'),
 item: 'apple',
 totalAmount: 36
}
```

## 5. db.sales.aggregate([{\$match: { status: "A" } }])

```
{
 _id: ObjectId('672c428ea88246916737a113'),
 item: 'apple',
 price: 1.2,
 quantity: 50,
 status: 'A'
}
{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
```

```
price: 0.8,
quantity: 100,
status: 'A'
}

{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 price: 0.8,
 quantity: 50,
 status: 'A'
}
```

```
6. db.sales.aggregate([{$count: "total_sales" }])
```

```
{
 total_sales: 5
}
```

Let's create another collection called inventory for this example.

**use SalesDB**

switched to db SalesDB

```
db["inventory"].find()
```

```
db.inventory.insertMany([
```

```
 { sku: "apple", stock: 150 },
```

```
 { sku: "banana", stock: 200 },
```

```
 { sku: "orange", stock: 100 }
])
```

```
{
```

```
{
```

```
_id: ObjectId('672c4a276b115129ae67f2dd'),
sku: 'apple',
stock: 150
}
{
_id: ObjectId('672c4a276b115129ae67f2de'),
sku: 'banana',
stock: 200
}
{
_id: ObjectId('672c4a276b115129ae67f2df'),
sku: 'orange',
stock: 100
}
```

```
7. db.sales.aggregate([
{
 $lookup: {
 from: "inventory",
 localField: "item",
 foreignField: "sku",
 as: "inventoryDetails"
 }
}
])
{
_id: ObjectId('672c428ea88246916737a113'),
item: 'apple',
price: 1.2,
```

```
quantity: 50,
status: 'A',
inventoryDetails: [
 {
 _id: ObjectId('672c4a276b115129ae67f2dd'),
 sku: 'apple',
 stock: 150
 }
]
}
{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
 price: 0.8,
 quantity: 100,
 status: 'A',
 inventoryDetails: [
 {
 _id: ObjectId('672c4a276b115129ae67f2de'),
 sku: 'banana',
 stock: 200
 }
]
}
{
 _id: ObjectId('672c428ea88246916737a115'),
 item: 'orange',
 price: 1.5,
```

```
quantity: 75,
status: 'B',
inventoryDetails: [
 {
 _id: ObjectId('672c4a276b115129ae67f2df'),
 sku: 'orange',
 stock: 100
 }
]
}
{
 _id: ObjectId('672c428ea88246916737a116'),
 item: 'apple',
 price: 1.2,
 quantity: 30,
 status: 'B',
 inventoryDetails: [
 {
 _id: ObjectId('672c4a276b115129ae67f2dd'),
 sku: 'apple',
 stock: 150
 }
]
}
{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 price: 0.8,
```

```
quantity: 50,
status: 'A',
inventoryDetails: [
 {
 _id: ObjectId('672c4a276b115129ae67f2de'),
 sku: 'banana',
 stock: 200
 }
]
}
```

```
8. db.sales.aggregate([
 { $match: { status: "A" } },
 { $out: "outCollection" }
])
```

- Filters documents with status: "A" and writes them to outCollection.

```
9. db.sales.aggregate([
 {
 $addFields: {
 totalAmount: { $multiply: ["$price", "$quantity"] }
 }
 }
])
{
 _id: ObjectId('672c428ea88246916737a113'),
 item: 'apple',
```

```
price: 1.2,
quantity: 50,
status: 'A',
totalAmount: 60
}

{
 _id: ObjectId('672c428ea88246916737a114'),
 item: 'banana',
 price: 0.8,
 quantity: 100,
 status: 'A',
 totalAmount: 80
}

{
 _id: ObjectId('672c428ea88246916737a115'),
 item: 'orange',
 price: 1.5,
 quantity: 75,
 status: 'B',
 totalAmount: 112.5
}

{
 _id: ObjectId('672c428ea88246916737a116'),
 item: 'apple',
 price: 1.2,
 quantity: 30,
 status: 'B',
 totalAmount: 36
```

```
}
{
 _id: ObjectId('672c428ea88246916737a117'),
 item: 'banana',
 price: 0.8,
 quantity: 50,
 status: 'A',
 totalAmount: 40
```

## Experiment No: 13

### 1. Objective: Search and Index.

#### Step 1: Creating a MongoDB Database and Collection

In MongoDB, you don't explicitly create a database. The database is created when you first store data in it.

```
use companyDB; // Switches to the database or creates it if it doesn't exist
```

```
db.createCollection("employees"); // Creates the "employees" collection
```

```
db.createCollection("articles"); // Creates the "articles" collection
```

Output:

```
{ "ok": 1 }
```

#### Step 2: Inserting Data into Collections

##### Employees Collection

```
db.employees.insertMany([
 { first_name: "John", last_name: "Doe", salary: 75000, date_of_birth: "1990-01-15",
 email: "john.doe@example.com" },
 { first_name: "Joan", last_name: "Smith", salary: 85000, date_of_birth: "1985-05-20",
 email: "joan.smith@example.com" }
]);
```

##### Articles Collection

```
db.articles.insertMany([
 { content: "MongoDB is a NoSQL database." },
 { content: "Databases like MongoDB are popular for modern web applications." }
]);
```

Output:

```
{ "acknowledged": true, "insertedIds": [ObjectId(...), ObjectId(...)] }
```

#### Step 3: Performing Searches

##### 1. Simple Search

**SQL:** SELECT \* FROM employees WHERE first\_name = 'John';

**MongoDB:**

```
db.employees.find({ first_name: "John" });
```

Output:

```
[
 { "_id": ObjectId("..."), "first_name": "John", "last_name": "Doe", "salary": 75000,
 "date_of_birth": "1990-01-15", "email": "john.doe@example.com" }
]
```

## 2. Wildcard Search

**SQL:** SELECT \* FROM employees WHERE first\_name LIKE 'Jo%';

**MongoDB:**

```
db.employees.find({ first_name: { $regex: "^Jo" } });
```

Output:

```
[
 { "_id": ObjectId("..."), "first_name": "John", "last_name": "Doe", ... },
 { "_id": ObjectId("..."), "first_name": "Joan", "last_name": "Smith", ... }
]
```

## 3. Full-Text Search

Full-text search in MongoDB requires creating a text index first.

**Create Full-Text Index:**

```
db.articles.createIndex({ content: "text" });
```

Output:

```
{ "createdCollectionAutomatically": false, "numIndexesBefore": 1,
 "numIndexesAfter": 2, "ok": 1 }
```

**Perform Full-Text Search:**

**SQL:** SELECT \* FROM articles WHERE MATCH(content) AGAINST('database');

**MongoDB:**

```
db.articles.find({ $text: { $search: "database" } });
```

Output:

```
[
 { "_id": ObjectId("..."), "content": "MongoDB is a NoSQL database." },
 { "_id": ObjectId("..."), "content": "Databases like MongoDB are popular for modern
 web applications." }
]
```

## 4. Range Search

**SQL:** SELECT \* FROM employees WHERE salary BETWEEN 50000 AND 100000;

**MongoDB:**

```
db.employees.find({ salary: { $gte: 50000, $lte: 100000 } });
```

Output:

```
[
 { "_id": ObjectId("..."), "first_name": "John", "last_name": "Doe", "salary": 75000, ...
},
 { "_id": ObjectId("..."), "first_name": "Joan", "last_name": "Smith", "salary": 85000,
 ... }
]
```

## Step 4: Indexing in MongoDB

### 1. Creating an Index

**SQL:** CREATE INDEX idx\_firstname ON employees(first\_name);

**MongoDB:**

```
db.employees.createIndex({ first_name: 1 });
```

Output:

```
{ "createdCollectionAutomatically": false, "numIndexesBefore": 1,
 "numIndexesAfter": 2, "ok": 1 }
```

### 2. Creating a Unique Index

**SQL:** CREATE UNIQUE INDEX idx\_unique\_email ON employees(email);

**MongoDB:**

```
db.employees.createIndex({ email: 1 }, { unique: true });
```

Output:

```
{ "createdCollectionAutomatically": false, "numIndexesBefore": 2,
 "numIndexesAfter": 3, "ok": 1 }
```

### 3. Composite Index

**SQL:** CREATE INDEX idx\_name\_dob ON employees(last\_name, date\_of\_birth);

**MongoDB:**

```
db.employees.createIndex({ last_name: 1, date_of_birth: 1 });
```

Output:

```
{ "createdCollectionAutomatically": false, "numIndexesBefore": 3,
 "numIndexesAfter": 4, "ok": 1 }
```

#### 4. Full-Text Index

**SQL:** CREATE FULLTEXT INDEX idx\_content ON articles(content);

**MongoDB:**

This step is the same as creating a text index shown earlier.

#### 5. Dropping an Index

**SQL:** DROP INDEX idx\_firstname ON employees;

**MongoDB:**

```
db.employees.dropIndex("first_name_1");
```

Output:

```
{ "nIndexesWas": 4, "ok": 1 }
```